■ Python Comprehensive MCQ Test Bank

CDAC PGCP-AI - Master Certification & Assessment Preparation

Target Audience: CDAC PGCP-AI & Advanced Python Developers	**Total Questions:** 300 Comprehensive MCQs & Code Snippets
Syllabus Scope: Complete Curriculum (Day 1 through Day 11)	**Structure:** 100 Basic - 100 Medium - 100 Advanced
Format: Multiple Choice, Output Prediction, Edge Cases	**Self-Testing Design:** Answers printed at bottom in small font

Curriculum Syllabus & Module Breakdown (Day 01 -> Day 11):

Days	Core Modules & Topics Covered	Target Focus
Day 1 - 2	Python Architecture, Bytecode/PVM, Syntax, Data Types, Operators (Floor Division, Modulo, Bitwise), Conditionals, Loops, Strings, Slicing, Methods & Formatting.	Foundations & Core Syntax
Day 3 - 4	Lists & Tuples, Mutability vs Immutability, Comprehensions, Dictionaries, Hashability, Sets, Views, Shallow vs Deep Copy.	Sequences & Data Structures
Day 5	Functions, *args/**kwargs, Positional/Keyword-Only, LEGB Scopes, Closures, Decorators, Lambdas, Built-ins (map/filter/reduce), Regular Expressions.	Functional & Text Processing
Day 6	OOP Foundations, Encapsulation, Mangling, Properties, Inheritance, Diamond Problem, MRO (C3 Linearization), super(), Dunders, ABCs, Descriptors.	Object-Oriented Design
Day 7	File I/O Streams, Seek/Tell Offsets, Context Managers, CSV DictReader, JSON Serialization, Pickle Security, SQLite Relational Database & Transactions.	Data Persistence & Relational DB
Day 8	Iterator Protocol (__iter__/__next__), StopIteration, Generators (yield, yield from), .send() Coroutines, Memory Efficiency, Exception Hierarchy.	Advanced Iteration & Generators
Day 9 - 10	Flask Web Basics, WSGI, Routing, Jinja2, REST API Architecture, HTTP Verbs, Idempotency, Status Codes (200, 201, 204, 404, 405), Web Scraping (BeautifulSoup).	Web Services & Scraping
Day 10 - 11	NumPy ndarray, Slicing, Shapes, Broadcasting, Boolean Indexing. Pandas Series & DataFrames, .loc/.iloc, Groupby Aggregations, Missing Data, Matplotlib OO Charts.	Data Analysis & Visualization

Exam & Self-Assessment Instructions:
- **Active Recall Mode:** The correct answers for each question are deliberately placed in a small, faint footer key at the very bottom of each page. Solve every question on paper first before checking.
- **Difficulty Progression:** Questions 1-100 establish core fundamentals; Questions 101-200 evaluate code tracing and tricky Python mechanics; Questions 201-300 challenge advanced concepts, internals, and edge cases.
- **End-of-Book Resources:** A unified 300-Question Master Answer Matrix and comprehensive Explanations are provided in the appendix.

SECTION 1: BASIC LEVEL (Questions 1 - 100)

Core Foundations: Python History, Syntax, Operators, Conditionals, Loops, Strings, Lists, Tuples, Dictionaries, Sets, Functions & Basic OOP

[Day 01: Python History & Origins]

Q1. Who created the Python programming language and in which year was it first released?

A) Guido van Rossum, 1991

B) James Gosling, 1995

C) Bjarne Stroustrup, 1985

D) Dennis Ritchie, 1972

[Day 01: Language Architecture]

Q2. How is Python source code (.py) executed by the standard CPython runtime?

A) It is purely interpreted line-by-line without any bytecode compilation stage.

B) It is translated into C code and compiled using GCC.

C) It is directly compiled into hardware machine code.

D) It is compiled into bytecode (.pyc) and then executed by the Python Virtual Machine (PVM).

[Day 01: Output Formatting]

Q3. What is the output of the following print statement?

```
print(&amp;quot;Python&amp;quot;, &amp;quot;CDAC&amp;quot;, &amp;quot;2026&amp;quot;, sep=&amp;quot;::&amp;quot;)
```

A) Python::CDAC::2026::

B) Python CDAC 2026

C) Python::CDAC::2026

D) Python, CDAC, 2026

[Day 01: Print Formatting]

Q4. What is printed by the following code snippet?

```
print(&amp;quot;Hello&amp;quot;, end=&amp;quot;***&amp;quot;)
print(&amp;quot;World&amp;quot;)
```

A) Hello\n***World

B) Hello***World

C) Hello World

D) Hello***\nWorld

[Day 01: Input Handling]

Q5. What is the return data type of Python's built-in input() function?

```
val = input(&amp;quot;Enter a number: &amp;quot;)
```

A) float if decimals are entered

B) str always

C) Based on dynamic type inference

D) int if only digits are entered

[Day 01: Identifiers & Naming Rules]

Q6. Which of the following is a valid variable identifier in Python?

A) _student_score_2

B) class

C) 2nd_record

D) total-amount

[Day 01: Floor Division]

Q7. What is the output of the following arithmetic expressions?

```
print(10 // 3, -10 // 3)
```

A) 3.0 -4.0

B) 3 -3

C) 3 -4

D) 3.33 -3.33

[Day 01: Division Operator]

Q8. What is the data type and value of the result of 12 / 4 in Python 3?

```
res = 12 / 4
print(type(res), res)
```

A) <class 'int'> 3

B) <class 'float'> 3.0

C) <class 'int'> 3.0

D) <class 'float'> 3

[Day 01: Exponentiation Operator]

Q9. What is the output of the following arithmetic expression?

```
print(2 ** 3 ** 2)
```

A) 512

B) 36

C) 128

D) 64

[Day 01: Modulo Operator]

Q10. What is the output of evaluating 17 % 5 in Python?

A) 3.4

B) 1

C) 3

D) 2

[Day 01: Identity vs Equality]

Q11. What is the fundamental difference between the '==' operator and the 'is' operator?

A) They are completely synonymous and interchangeable.

B) '==' compares memory addresses, whereas 'is' compares values.

C) '==' compares values/equality, whereas 'is' compares object identity (memory addresses).

D) '==' is used only for numbers, while 'is' is used only for strings.

[Day 01: Membership Operators]

Q12. What is the output of the following code snippet?

```
fruits = [&amp;quot;apple&amp;quot;, &amp;quot;banana&amp;quot;, &amp;quot;cherry&amp;quot;]
print(&amp;quot;ban&amp;quot; in fruits, &amp;quot;banana&amp;quot; in fruits)
```

A) True True

B) False True

C) True False

D) False False

[Day 01: Type Casting]

Q13. Which of the following type conversions will raise a ValueError?

A) float('3.14')

B) int('45.67')

C) bool('False')

D) int('100')

[Day 01: Boolean Evaluation]

Q14. What is the output of bool('False') in Python?

```
print(bool(&amp;quot;False&amp;quot;), bool(&amp;quot;&amp;quot;))
```

A) False True

B) True True

C) False False

D) True False

[Day 01: Ternary Operator]

Q15. What is the correct syntax for Python's conditional (ternary) expression?

A) value_if_true when condition otherwise value_if_false

B) condition ? value_if_true : value_if_false

C) value_if_true if condition else value_if_false

D) if condition then value_if_true else value_if_false

[Day 01: Range Function]

Q16. What sequence of integers is generated by list(range(2, 9, 3))?

A) [2, 5, 8]

B) [2, 5, 8, 11]

C) [3, 6, 9]

D) [2, 3, 4, 5, 6, 7, 8]

[Day 01: Range Defaults]

Q17. What is produced by list(range(5))?

A) [0, 1, 2, 3, 4]

B) [0, 1, 2, 3, 4, 5]

C) [5]

D) [1, 2, 3, 4, 5]

[Day 01: Loop Control Statements]

Q18. What does the break statement do when encountered inside a loop?

A) Restarts the loop from the first iteration.

B) Exits the entire Python program.

C) Skips the current iteration and jumps to the next iteration.

D) Immediately terminates the innermost loop.

[Day 01: Continue Statement]

Q19. What is the output of the following code snippet?

```
total = 0
for i in range(5):
    if i % 2 == 0:
        continue
    total += i
print(total)
```

A) 0

B) 10

C) 6

D) 4

[Day 01: For-Else Construct]

Q20. When does the else block attached to a for or while loop execute?

A) Only if the loop was terminated prematurely by a break statement.

B) Only if the loop completes normally without encountering a break statement.

C) At the start of every iteration.

D) Only if an unhandled exception occurs inside the loop.

[Day 01: Pass Statement]

Q21. What is the purpose of the pass statement in Python?

A) It acts as a null statement / placeholder where code is syntactically required but no action is needed.

B) It forces the interpreter to skip garbage collection.

C) It passes execution to the next function.

D) To pass a variable to another scope.

[Day 01: Logical Operators]

Q22. What is the output of evaluating not (5 > 3 and 2 == 1)?

A) None

B) 1

C) False

D) True

[Day 01: Bitwise Operators]

Q23. What is the result of the bitwise operation 5 & 3 in Python?

A) 2

B) 7

C) 1

D) 8

[Day 01: Bitwise XOR]

Q24. What is the output of 6 ^ 3 in Python?

A) 216

B) 5

C) 9

D) 7

[Day 01: Built-in type() and id()]

Q25. Which built-in function returns the unique integer memory identity of a Python object?

A) id()

B) ref()

C) hash()

D) type()

[Day 02: String Immutability]

Q26. What happens when you execute the following code?

```
s = &amp;quot;Python&amp;quot;
s[0] = &amp;quot;J&amp;quot;
```

A) ValueError: strings cannot be modified

B) SyntaxError: invalid assignment

C) s becomes 'Jython'

D) TypeError: 'str' object does not support item assignment

[Day 02: Basic Slicing]

Q27. What is the output of text[1:5] where text = 'PROGRAM'?

```
text = &amp;quot;PROGRAM&amp;quot;
print(text[1:5])
```

A) ROGRA

B) PROG

C) ROGR

D) ROCR

[Day 02: Negative Slicing]

Q28. What is the output of text[-3:] where text = 'DEVELOPER'?

```
text = &amp;quot;DEVELOPER&amp;quot;
print(text[-3:])
```

A) PER

B) OPE

C) LOP

D) REP

[Day 02: String Reversal]

Q29. Which of the following expressions reverses a string s using slice notation?

A) s[::-1]

B) s[-1:0]

C) s[1:len(s):-1]

D) s[0:-1]

[Day 02: String Stripping]

Q30. What does the strip() method do on a string?

```
s = &quot;  hello world  \n&quot;
print(repr(s.strip()))
```

A) Removes only trailing newlines.

B) Converts the string to lowercase.

C) Removes all spaces from everywhere inside the string.

D) Removes leading and trailing whitespace characters.

[Day 02: String Splitting]

Q31. What is the output of s.split(',') when s = 'apple,banana,orange'?

```
s = &quot;apple,banana,orange&quot;
print(s.split(&quot;,&quot;))
```

A) 'apple banana orange'

B) ('apple', 'banana', 'orange')

C) ['apple', 'banana', 'orange']

D) {'apple', 'banana', 'orange'}

[Day 02: String Joining]

Q32. What is the output of the following code snippet?

```
words = [&quot;AI&quot;, &quot;ML&quot;, &quot;DL&quot;]
print(&quot;-&quot;.join(words))
```

A) AI ML DL

B) AI-ML-DL

C) -AI-ML-DL-

D) ['AI', 'ML', 'DL']

[Day 02: find() vs index()]

Q33. What happens when you search for a substring that is NOT found using find() vs index()?

A) Both raise a ValueError.

B) find() returns -1, while index() raises a ValueError.

C) find() raises ValueError, while index() returns None.

D) Both return -1.

[Day 02: String Case Methods]

Q34. What is the output of 'python programming'.title()?

A) 'PYTHON PROGRAMMING'

B) 'Python Programming.'

C) 'Python programming'

D) 'Python Programming'

[Day 02: Raw Strings]

Q35. What is the output of printing a raw string r'C:\new\test'?

```
path = r&quot;C:\new\test&quot;
print(path)
```

A) SyntaxError

B) C: followed by a newline then ew\test

C) C:\new\test

D) C:\new followed by a tab

[Day 02: String Formatting (f-strings)]

Q36. What is the output of the following f-string with a float precision specifier?

```
pi = 3.1415926
print(f&quot;Pi is {pi:.2f}&quot;)
```

A) Pi is 3.1415926

B) Pi is 3.14

C) Pi is 3.15

D) Pi is 3.1

[Day 02: String replace()]

Q37. What is the output of the following code snippet?

```
msg = &quot;java is cool&quot;
msg.replace(&quot;java&quot;, &quot;python&quot;)
print(msg)
```

A) java is cool

B) None

C) TypeError

D) python is cool

[Day 02: String startswith()]

Q38. What is the return value of 'https://cdac.in'.startswith(('http://', 'https://'))?

A) TypeError: startswith does not take a tuple

B) None

C) True

D) False

[Day 02: String Character Check]

Q39. What is the output of '1234a'.isdigit() and '1234'.isdigit()?

A) False False

B) True True

C) False True

D) True False

[Day 02: String Repetition]

Q40. What is the output of '=' * 5?

A) TypeError: cannot multiply string by int

B) '====='

C) '5'

D) '== = = ='

[Day 02: String count()]

Q41. What is the output of 'banana'.count('an')?

A) 2

B) 3

C) 0

D) 1

[Day 02: String Multiline]

Q42. How are multiline string literals defined in Python?

A) Using the multiline keyword

B) Ending every line with a comma

C) Enclosing in single backticks `...`

D) Enclosing in triple single ('''...''') or triple double ("""...""") quotes

[Day 02: String Concatenation]

Q43. What is the output of 'Hello ' + 'World' vs 'Hello ' * 2?

A) 'HelloWorld' and 'HelloHello'

B) 'HelloWorld' and 'Hello Hello '

C) 'Hello World' and 'Hello Hello '

D) 'Hello World' and 'HelloHello'

[Day 02: len() Function]

Q44. What is the output of len('Hello\nWorld')?

A) 12

B) 11

C) 10

D) 13

[Day 02: String Formatting (% vs format)]

Q45. What is the output of 'Item: %s, Price: %.1f' % ('Book', 19.95)?

A) Item: Book, Price: 20.0

B) Item: Book, Price: 19.9

C) Item: Book, Price: 20

D) Item: Book, Price: 19.95

[Day 03: List Mutability]

Q46. Which of the following operations on a Python list is valid?

```
nums = [10, 20, 30]
nums[1] = 99
```

A) Raises IndexError

B) Appends 99 to the end of the list

C) Raises TypeError because lists are immutable

D) Modifies the list to [10, 99, 30]

[Day 03: Single Element Tuple]

Q47. Which of the following correctly defines a single-element tuple containing the integer 42?

A) t = {42}

B) t = (42)

C) t = [42]

D) t = (42,)

[Day 03: list.append() vs list.extend()]

Q48. What is the output of the following code snippet?

```
a = [1, 2]
a.append([3, 4])
b = [1, 2]
b.extend([3, 4])
print(len(a), len(b))
```

A) 4 4

B) 3 4

C) 3 3

D) 4 3

[Day 03: list.insert()]

Q49. What is the output of list.insert(1, 'X') on nums = ['A', 'B', 'C']?

```
nums = [&#x27;A&#x27;, &#x27;B&#x27;, &#x27;C&#x27;]
nums.insert(1, &#x27;X&#x27;)
print(nums)
```

A) ['A', 'X', 'B', 'C']

B) ['A', 'B', 'X', 'C']

C) ['A', 'X', 'C']

D) ['X', 'A', 'B', 'C']

[Day 03: list.pop()]

Q50. What does list.pop() without arguments do, and what does it return?

```
nums = [10, 20, 30]
val = nums.pop()
print(val, nums)
```

A) Returns None and empties the list

B) Raises an error because index argument is required

C) Removes and returns the first element (10)

D) Removes and returns the last element (30)

[Day 03: list.remove()]

Q51. What happens when you call nums.remove(99) if 99 is not in nums = [1, 2, 3]?

A) It does nothing silently

B) It returns -1

C) It returns False

D) It raises a ValueError

[Day 03: Tuple Immutability]

Q52. What is the output of attempting to modify a tuple?

```
t = (1, 2, 3)
t[0] = 10
```

A) t becomes (10, 2, 3)

B) TypeError: 'tuple' object does not support item assignment

C) ValueError: tuples cannot be changed

D) AttributeError: tuple has no set attribute

[Day 03: Tuple Unpacking]

Q53. What is the output of the following tuple unpacking statement?

```
a, b, c = (10, 20, 30)
print(b)
```

A) 20

B) 30

C) (10, 20)

D) 10

[Day 03: Variable Swap]

Q54. How does Python swap two variables x and y in a single line without a temporary variable?

A) x, y = y, x

B) x = y; y = x

C) x.swap(y)

D) swap(x, y)

[Day 03: List Comprehension Basics]

Q55. What does the list comprehension [x * 2 for x in range(4)] produce?

A) [0, 2, 4, 6, 8]

B) [0, 2, 4, 6]

C) [2, 4, 6, 8]

D) [0, 1, 2, 3]

[Day 03: List Comprehension with Filter]

Q56. What is the output of [x for x in range(10) if x % 3 == 0]?

A) [3, 6, 9]

B) [0, 3, 6, 9]

C) [0, 3, 6, 9, 12]

D) [1, 3, 6, 9]

[Day 03: list.sort() vs sorted()]

Q57. What is the difference between nums.sort() and sorted(nums)?

A) nums.sort() sorts in place and returns None; sorted(nums) returns a new sorted list.

B) sorted(nums) modifies in place; nums.sort() returns a new list.

C) sorted() works only on numbers; sort() works on all types.

D) Both sort the list in place and return None.

[Day 03: list.index()]

Q58. What is the output of [10, 20, 30, 20].index(20)?

A) [1, 3]

B) 2

C) 1

D) 3

[Day 03: del Statement]

Q59. What is the output of the following code snippet?

```
nums = [1, 2, 3, 4]
del nums[1:3]
print(nums)
```

A) []

B) [1, 4]

C) [1, 2, 4]

D) [2, 3]

[Day 03: Built-in min, max, sum]

Q60. What is the output of print(min(nums), max(nums), sum(nums)) where nums = [4, 1, 7, 2]?

A) 1 7 14

B) 1 7 12

C) 4 7 14

D) 1 4 14

[Day 04: Dictionary Key Requirements]

Q61. Which of the following data types CANNOT be used as a key in a Python dictionary?

A) int

B) tuple of strings

C) list

D) str

[Day 04: Dictionary Access (.get)]

Q62. What happens when you access a non-existent key using d['unknown'] vs d.get('unknown', 0)?

A) d['unknown'] raises KeyError, while d.get('unknown', 0) returns 0.

B) d['unknown'] returns None, while d.get() raises KeyError.

C) Both raise KeyError.

D) Both return 0.

[Day 04: dict.keys() & dict.values()]

Q63. What is the output of the following dictionary operations?

```
d = {&amp;quot;a&amp;quot;: 1, &amp;quot;b&amp;quot;: 2}
print(list(d.keys()), list(d.values()))
```

A) {'a', 'b'} {1, 2}

B) ['a', 'b'] [1, 2]

C) [1, 2] ['a', 'b']

D) [('a', 1), ('b', 2)]

[Day 04: dict.pop()]

Q64. What is the output of data.pop('c', 99) where data = {'a': 10, 'b': 20}?

```
data = {&amp;quot;a&amp;quot;: 10, &amp;quot;b&amp;quot;: 20}
res = data.pop(&amp;quot;c&amp;quot;, 99)
print(res)
```

A) KeyError

B) None

C) 99

D) {'a': 10, 'b': 20}

[Day 04: dict.update()]

Q65. What does d1.update(d2) do?

```
d1 = {&amp;quot;a&amp;quot;: 1, &amp;quot;b&amp;quot;: 2}
d2 = {&amp;quot;b&amp;quot;: 99, &amp;quot;c&amp;quot;: 3}
d1.update(d2)
print(d1)
```

A) {'a': 1, 'b': 99, 'c': 3}

B) {'b': 99, 'c': 3}

C) Raises KeyError on duplicate key 'b'

D) {'a': 1, 'b': 2, 'c': 3}

[Day 04: Empty Set vs Empty Dict]

Q66. What are the data types of obj1 = {} and obj2 = set()?

```
obj1 = {}
obj2 = set()
```

A) obj1 is dict, obj2 is set

B) Both are dicts

C) Both are sets

D) obj1 is set, obj2 is dict

[Day 04: Set Uniqueness]

Q67. What is the output of len(set([1, 2, 2, 3, 3, 3, 4]))?

A) 1

B) 7

C) 4

D) 3

[Day 04: Set Operations (Union)]

Q68. What is the result of {1, 2, 3} | {3, 4, 5} in Python?

A) {3}

B) {1, 2, 3, 4, 5}

C) {1, 2, 4, 5}

D) [1, 2, 3, 4, 5]

[Day 04: Set Operations (Intersection)]

Q69. What is the result of {1, 2, 3} & {2, 3, 4}?

A) {2, 3}

B) {1, 2, 3, 4}

C) set()

D) {1, 4}

[Day 04: Set Operations (Difference)]

Q70. What is the result of {1, 2, 3, 4} - {2, 4, 6}?

A) {1, 3, 6}

B) {2, 4}

C) {1, 3}

D) {6}

[Day 04: set.discard() vs set.remove()]

Q71. What happens when you call s.discard(99) vs s.remove(99) on s = {1, 2, 3}?

A) Both return False.

B) Both raise KeyError.

C) remove() raises KeyError, while discard() does nothing and raises no error.

D) discard() raises KeyError, while remove() does nothing.

[Day 04: Dictionary Comprehension]

Q72. What does the dictionary comprehension {x: x2 for x in range(3)} produce?**

A) {0: 0, 1: 1, 2: 4}

B) {1: 1, 2: 4, 3: 9}

C) [0, 1, 4]

D) {(0, 0), (1, 1), (2, 4)}

[Day 04: Iterating Dictionary]

Q73. When iterating over a dictionary using 'for item in d:', what does 'item' represent?

```
d = {&amp;quot;x&amp;quot;: 10, &amp;quot;y&amp;quot;: 20}
for item in d:
    print(item)
```

A) The keys: 'x', 'y'

B) Tuples of (key, value): ('x', 10), ('y', 20)

C) A single list of all items

D) The values: 10, 20

[Day 04: dict.items()]

Q74. What does dict.items() yield during iteration?

A) (key, value) 2-element tuples

B) Sub-dictionaries

C) Only the keys

D) Only the values

[Day 04: Frozenset]

Q75. What is a frozenset in Python?

A) A set that cannot be iterated over

B) A set stored on disk in frozen binary format

C) An immutable, hashable version of a set that can be used as a dictionary key

D) A set that only accepts frozen floating point numbers

[Day 05: Function Default Return]

Q76. What is the return value of a Python function that executes without reaching an explicit return statement?

```
def greet():
    print(&amp;quot;Hello&amp;quot;)
res = greet()
print(res)
```

A) 0

B) False

C) None

D) '' (empty string)

[Day 05: *args and **kwargs]

Q77. In a function definition 'def func(*args, **kwargs):', what are the types of args and kwargs?

A) args is a tuple, kwargs is a dict

B) args is a dict, kwargs is a list

C) Both are lists

D) args is a list, kwargs is a tuple

[Day 05: Default Parameters]

Q78. Which of the following function definitions is syntactically INVALID in Python?

A) def calc(a=10, b): pass

B) def calc(a, b, c=20): pass

C) def calc(a, b=10): pass

D) def calc(a=5, b=10): pass

[Day 05: Lambda Syntax]

Q79. Which of the following correctly defines an anonymous lambda function that squares a number?

A) square = lambda(x) { return x ** 2; }

B) square = lambda x -> x ** 2

C) square = lambda x: x ** 2

D) square = def(x): return x ** 2

[Day 05: Built-in map()]

Q80. What is the output of list(map(lambda x: x + 1, [1, 2, 3]))?

A) [1, 2, 3]

B) [2, 3, 4]

C) [1, 4, 9]

D) [2, 4, 6]

[Day 05: Built-in filter()]

Q81. What does list(filter(lambda x: x > 0, [-2, 0, 3, -1, 5])) return?

A) [3, 5]

B) [0, 3, 5]

C) [True, False, True, False, True]

D) [-2, -1]

[Day 05: Built-in zip()]

Q82. What is the output of list(zip(['a', 'b'], [1, 2]))?

A) ['a', 1, 'b', 2]

B) [['a', 1], ['b', 2]]

C) [{'a': 1}, {'b': 2}]

D) [('a', 1), ('b', 2)]

[Day 05: Built-in enumerate()]

Q83. What does list(enumerate(['apple', 'banana'])) return?

A) [0, 1]

B) [(0, 'apple'), (1, 'banana')]

C) [(1, 'apple'), (2, 'banana')]

D) {'apple': 0, 'banana': 1}

[Day 05: global Keyword]

Q84. Why is the global keyword used inside a Python function?

```
x = 10
def modify():
    global x
    x = 20
```

A) To make the function accessible in other files

B) To allow binding/modifying a module-level variable from inside local function scope

C) To prevent other threads from modifying x

D) To optimize performance

[Day 05: Regular Expressions (re.search vs re.match)]

Q85. What is the key difference between re.match() and re.search() in Python's re module?

A) re.search() matches only at the beginning; re.match() scans anywhere.

B) re.match() returns all matches; re.search() returns only the first match.

C) They are identical.

D) re.match() matches only at the beginning of the string; re.search() scans the entire string.

[Day 05: RegEx Metacharacters]

Q86. In regular expressions, which metacharacter matches any single digit character (equivalent to [0-9])?

A) \s

B) \b

C) \w

D) \d

[Day 05: RegEx findall()]

Q87. What is the return type of re.findall(r'\d+', 'Apples: 10, Oranges: 25')?

A) An iterator

B) A match object

C) A list of strings: ['10', '25']

D) A tuple of integers: (10, 25)

[Day 05: Built-in all() and any()]

Q88. What is the output of print(any([False, 0, '']), all([True, 1, 'Python']))?

A) False True

B) True True

C) False False

D) True False

[Day 06: OOP self Parameter]

Q89. In a Python class method, what does the 'self' parameter represent?

A) An explicit reference to the current instance of the class

B) A global pointer to the parent class

C) The memory address of the module

D) A keyword referring to the class definition

[Day 06: OOP Initializer __init__]

Q90. What is the role of the special __init__ method in a Python class?

A) It acts as the initializer/constructor to set up attributes on a newly created instance

B) To convert a class into a string

C) To destroy an object and free its memory

D) To allocate raw memory for a new object

[Day 06: Instance vs Class Variables]

Q91. What is the output of the following code snippet?

```
class Student:
    school = &amp;quot;CDAC&amp;quot;
    def __init__(self, name):
        self.name = name
s1 = Student(&amp;quot;Alice&amp;quot;)
s2 = Student(&amp;quot;Bob&amp;quot;)
print(s1.school, s2.school)
```

A) AttributeError

B) Alice Bob

C) CDAC CDAC

D) None None

[Day 06: Basic Inheritance]

Q92. How does class Dog inherit from class Animal in Python?

A) class Dog extends Animal:

B) class Dog(Animal):

C) class Dog inherits Animal:

D) class Dog : public Animal:

[Day 06: Built-in isinstance()]

Q93. What is the output of isinstance(5, (str, int, float))?

A) False

B) TypeError

C) <class 'int'>

D) True

[Day 06: String Representation __str__]

Q94. Which dunder method is invoked when you pass an object to print() or str()?

A) __print__

B) __display__

C) __repr__

D) __str__

[Day 07: File open() Default Mode]

Q95. What is the default access mode when opening a file using Python's open('data.txt')?

A) 'rb' (read binary)

B) 'w' (write)

C) 'r' (read text)

D) 'a' (append)

[Day 07: Context Manager 'with']

Q96. What is the primary advantage of using the 'with' statement when opening files in Python?

A) It makes file reads run 10x faster.

B) It automatically closes the file stream when the block exits, even if exceptions occur.

C) It encrypts the contents on disk.

D) It allows writing to read-only files.

[Day 07: JSON Serialization]

Q97. Which json module function converts a Python dictionary directly into a JSON formatted string?

A) json.dumps()

B) json.load()

C) json.loads()

D) json.dump()

[Day 09: Flask Application Setup]

Q98. Which decorator in Flask binds a view function to an incoming HTTP URL endpoint?

A) @app.bind()
B) @app.endpoint()
C) @app.url()
D) @app.route()

[Day 10: HTTP Status Codes]

Q99. Which HTTP status code signifies that a request succeeded and resulted in the creation of a new resource?

A) 301 Moved Permanently
B) 200 OK
C) 201 Created
D) 204 No Content

[Day 11: Pandas DataFrame Inspection]

Q100. Which Pandas DataFrame method displays the first 5 rows by default?

A) df.tail()
B) df.first()
C) df.head()
D) df.preview()

SECTION 2: MEDIUM LEVEL (Questions 101 - 200)

Intermediate Mechanics: Tricky Slicing, Short-Circuit Logic, Comprehensions, Scopes, Closures, Decorators, Properties, File Streams, SQLite & REST APIs

[Day 01: Short-Circuit Evaluation]

Q101. What is the output of the following Python code demonstrating short-circuit evaluation?

```
a = &amp;quot;&amp;quot; or &amp;quot;Python&amp;quot;
b = 0 and 100
c = [1, 2] or []
print(a, b, c)
```

A) Python 0 [1, 2]

B) Python 100 [1, 2]

C) "" 0 []

D) True False True

[Day 02: Negative Stride Slicing]

Q102. Given the string text = 'PYTHON', what is the output of print(text[4:1:-1])?

```
text = &amp;quot;PYTHON&amp;quot;
print(text[4:1:-1])
```

A) NOH

B) THO

C) OHT

D) OHTY

[Day 02: String Immutability in Pipelines]

Q103. What is the output of the following Python code demonstrating string immutability?

```
s = &amp;quot;python programming&amp;quot;
s.upper()
s.replace(&amp;quot;programming&amp;quot;, &amp;quot;development&amp;quot;)
res = s.title()
print(res)
```

A) PYTHON PROGRAMMING

B) PYTHON DEVELOPMENT

C) Python Development

D) Python Programming

[Day 01: Loop Else with Break]

Q104. What is the output of the following loop construct?

```
for n in range(2, 6):
    if n % 4 == 0:
        print(&amp;quot;Found&amp;quot;, n)
        break
else:
    print(&amp;quot;Not found&amp;quot;)
```

A) Found 4\nNot found

B) Found 4

C) Not found

D) Found 2

[Day 01: Chained Comparison Operators]

Q105. What does Python evaluate the chained expression 1 < 3 > 2 == 2 to?

```
res = 1 &amp;lt; 3 &amp;gt; 2 == 2
print(res)
```

A) False

B) TypeError

C) 2

D) True

[Day 01: Bitwise Shift Operations]

Q106. What is the result of evaluating (8 >> 2) + (3 << 2) in Python?

A) 12

B) 10

C) 14

D) 16

[Day 02: String find() with Start Index]

Q107. What is the output of the following string find method call?

```
text = &amp;quot;abracadabra&amp;quot;
print(text.find(&amp;quot;a&amp;quot;, 4))
```

A) 7

B) 0

C) 3

D) 5

[Day 02: String split() with maxsplit]

Q108. What is the output of 'a:b:c:d'.split(':', 2)?

A) ['a', 'b', 'c:d']

B) ['a', 'b', 'c', 'd']

C) ['a:b', 'c:d']

D) ['a', 'b:c:d']

[Day 02: String Stripping with Specific Characters]

Q109. What is the output of 'www.cdac.in'.strip('wn.')?

A) 'www.cdac.i'

B) '.cdac.'

C) 'cdac.in'

D) 'cdac'

[Day 01: Integer Caching / Small Integer Interning]

Q110. What is the output of the following comparison demonstrating integer caching in CPython?

```
x = 256
y = 256
a = 257
b = 257
print(x is y, a == b)
```

A) True False

B) False False

C) True True

D) False True

[Day 02: String zfill and rjust]

Q111. What is the output of '42'.zfill(5)?

A) '0042'

B) '42000'

C) '00042'

D) '42.000'

[Day 01: Operator Precedence]

Q112. What is the output of 3 + 4 * 2 > 10 and not 2 == 3?

A) False

B) True

C) 11

D) None

[Day 02: f-string Expression Specifier]

Q113. What does the f-string debugging syntax f'{x=}' output when x = 10?

```
x = 10
print(f&amp;quot;{x=}&amp;quot;)
```

A) x=10

B) {x=10}

C) SyntaxError

D) 10

[Day 01: While Loop Decrement]

Q114. What does the following while loop print?

```
i = 3
while i &amp;gt; 0:
    i -= 1
    print(i, end=&amp;quot; &amp;quot;)
```

A) 3 2 1 0

B) 2 1

C) 3 2 1

D) 2 1 0

[Day 02: String partition()]

Q115. What is the output of 'student@cdac.in'.partition('@')?

A) 'student'

B) ['student', 'cdac.in']

C) ('student', '@', 'cdac.in')

D) ('student', 'cdac.in')

[Day 03: Shallow Copy Mechanics]

Q116. Consider the following Python code: a = [[1, 2], [3, 4]] b = a.copy() b[0][1] = 99 b.append([5, 6]) print(len(a), a[0][1]) What is the output?

```
a = [[1, 2], [3, 4]]
b = a.copy()
b[0][1] = 99
b.append([5, 6])
print(len(a), a[0][1])
```

A) 2 99

B) 3 99

C) 2 2

D) 3 2

[Day 03: Nested List Comprehension]

Q117. What is the output of the following nested list comprehension?

```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
evens = [x for row in matrix for x in row if x % 2 == 0]
print(evens)
```

A) [2, 4, 6, 8]

B) [4, 6, 8]

C) [1, 3, 5, 7, 9]

D) [[2], [4, 6], [8]]

[Day 03: List Multiplication Pitfall]

Q118. What is the output of the following code creating a 2D list via list multiplication?

```
grid = [[0] * 2] * 2
grid[0][0] = 5
print(grid)
```

A) [[5, 5], [5, 5]]

B) [[0, 5], [0, 5]]

C) [[5, 0], [0, 0]]

D) [[5, 0], [5, 0]]

[Day 04: Dictionary Dynamic Views]

Q119. Consider the following dictionary code: inventory = {'pen': 10, 'notebook': 5} keys = inventory.keys() inventory['pencil'] = 8 inventory.pop('pen') print(list(keys)) What is the output?

```
inventory = {&amp;quot;pen&amp;quot;: 10, &amp;quot;notebook&amp;quot;: 5}
keys = inventory.keys()
inventory[&amp;quot;pencil&amp;quot;] = 8
inventory.pop(&amp;quot;pen&amp;quot;)
print(list(keys))
```

A) RuntimeError is raised

B) ['pen', 'notebook']

C) ['notebook', 'pencil']

D) ['pen', 'notebook', 'pencil']

[Day 04: dict.setdefault()]

Q120. What is the output of the following code using dict.setdefault()?

```
data = {&amp;quot;a&amp;quot;: 1}
x = data.setdefault(&amp;quot;a&amp;quot;, 100)
y = data.setdefault(&amp;quot;b&amp;quot;, 200)
print(x, y, data[&amp;quot;b&amp;quot;])
```

A) 100 200 200

B) 1 200 200

C) 1 200 None

D) KeyError

[Day 04: Dictionary Merge / Unpacking]

Q121. What is the output of merging two dictionaries using the ** unpacking operator?

```
d1 = {&amp;quot;x&amp;quot;: 1, &amp;quot;y&amp;quot;: 2}
d2 = {&amp;quot;y&amp;quot;: 99, &amp;quot;z&amp;quot;: 3}
res = {**d1, **d2}
print(res)
```

A) {'x': 1, 'y': 99, 'z': 3}

B) {'x': 1, 'y': [2, 99], 'z': 3}

C) SyntaxError

D) {'x': 1, 'y': 2, 'z': 3}

[Day 04: Dictionary fromkeys() Pitfall]

Q122. What is the output of dict.fromkeys(['a', 'b'], []) after modifying one list?

```
d = dict.fromkeys([&amp;quot;a&amp;quot;, &amp;quot;b&amp;quot;], [])
d[&amp;quot;a&amp;quot;].append(1)
print(d)
```

A) {'a': [], 'b': []}

B) KeyError

C) {'a': [1], 'b': []}

D) {'a': [1], 'b': [1]}

[Day 03: Modifying List During Iteration]

Q123. What is the output of removing items from a list while iterating over it with a for loop?

```
nums = [1, 2, 3, 4, 5]
for x in nums:
    if x % 2 != 0:
        nums.remove(x)
print(nums)
```

A) []

B) [2, 4]

C) [2, 4, 5]

D) [1, 3, 5]

[Day 03: Tuple with Mutable List]

Q124. What happens when you append to a list inside a tuple?

```
t = (1, [2, 3])
t[1].append(4)
print(t)
```

A) TypeError: 'tuple' object does not support item assignment

B) (1, [2, 3, 4])

C) (1, [2, 3])

D) ValueError

[Day 04: Set Symmetric Difference]

Q125. What is the output of {1, 2, 3} ^ {2, 3, 4} in Python?

A) {1, 4}

B) {1, 2, 3, 4}

C) set()

D) {2, 3}

[Day 03: List Extended Slicing Assignment]

Q126. What is the output of the following slice assignment?

```
nums = [1, 2, 3, 4, 5]
nums[1:4] = [20, 30]
print(nums)
```

A) [1, 20, 30, 4, 5]

B) ValueError

C) [1, 20, 30, 5]

D) [1, [20, 30], 5]

[Day 03: list.sort() with Key Function]

Q127. What is the output of sorting words by length?

```
words = [&amp;quot;banana&amp;quot;, &amp;quot;pie&amp;quot;, &amp;quot;apple&amp;quot;]
words.sort(key=len)
print(words)
```

A) ['pie', 'banana', 'apple']

B) ['apple', 'banana', 'pie']

C) ['pie', 'apple', 'banana']

D) ['banana', 'apple', 'pie']

[Day 04: Dictionary Popitem]

Q128. What does d.popitem() do on a Python 3.7+ dictionary?

```
d = {&amp;quot;a&amp;quot;: 1, &amp;quot;b&amp;quot;: 2, &amp;quot;c&amp;quot;: 3}
item = d.popitem()
print(item)
```

A) Removes and returns an arbitrary (random) key-value pair

B) Removes and returns the last inserted key-value pair as a tuple ('c', 3)

C) Removes and returns the first inserted pair ('a', 1)

D) Raises TypeError if no arguments are passed

[Day 03: list.count() vs in]

Q129. What is the output of the following list operations?

```
nums = [True, 1, 0, False]
print(nums.count(1), nums.count(0))
```

A) 2 2

B) 1 2

C) 2 1

D) 1 1

[Day 04: Set issubset & issuperset]

Q130. What is the output of {1, 2}.issubset({1, 2, 3}) and {1, 2, 3}.issuperset({1, 2})?

A) False True

B) False False

C) True True

D) True False

[Day 05: Mutable Default Argument Pitfall]

Q131. Consider the following function defined with a mutable default argument: def append_record(item, registry=[]): registry.append(item) return registry list1 = append_record('CPU') list2 = append_record('GPU', []) list3 = append_record('RAM') print(list1 is list3, list1, list2) What is the output?

```
def append_record(item, registry=[]):
    registry.append(item)
    return registry
list1 = append_record(&amp;quot;CPU&amp;quot;)
list2 = append_record(&amp;quot;GPU&amp;quot;, [])
list3 = append_record(&amp;quot;RAM&amp;quot;)
print(list1 is list3, list1, list2)
```

A) True ['CPU'] ['GPU']

B) False ['CPU'] ['GPU']

C) True ['CPU', 'RAM'] ['GPU']

D) False ['CPU', 'RAM'] ['GPU']

[Day 05: Higher-Order Functions (map + filter)]

Q132. What is the output of the following Python code utilizing built-in higher-order functions?

```
nums = [1, 2, 3, 4, 5, 6]
res = list(map(lambda x: x * 10, filter(lambda x: x % 2 != 0, nums)))
print(res)
```

A) [20, 40, 60]

B) [10, 30, 50]

C) [10, 20, 30, 40, 50, 60]

D) [1, 3, 5]

[Day 05: Regular Expressions match vs search]

Q133. What is the output of the following regular expression matching code?

```
import re
text = &amp;quot;Order ID: 98452 shipped&amp;quot;
m1 = re.match(r&amp;quot;\d+&amp;quot;, text)
m2 = re.search(r&amp;quot;\d+&amp;quot;, text)
print(m1 is None, m2.group())
```

A) True 98452

B) True Order

C) False Order

D) False 98452

[Day 05: Keyword-Only Arguments (*)]

Q134. In the function definition 'def connect(host, port, *, timeout=30):', how must 'timeout' be passed?

A) Strictly as a keyword argument (e.g. timeout=10)

B) It cannot be passed at all

C) As a positional argument only

D) As either positional or keyword argument

[Day 05: Positional-Only Arguments (/)]

Q135. What is the purpose of the slash '/' in 'def add(a, b, /, c):'?

A) It is a syntax error

B) Performs division on a and b

C) Specifies that a and b must be passed as positional-only arguments

D) Specifies that c must be passed as positional-only

[Day 05: nonlocal Keyword]

Q136. What is the output of the following nested function with nonlocal?

```
def outer():
    x = 10
    def inner():
        nonlocal x
        x += 5
    inner()
    return x
print(outer())
```

A) 10

B) 15

C) UnboundLocalError

D) None

[Day 05: functools.reduce()]

Q137. What is the output of functools.reduce(lambda acc, x: acc * x, [1, 2, 3, 4])?

```
from functools import reduce
res = reduce(lambda acc, x: acc * x, [1, 2, 3, 4])
print(res)
```

A) 24

B) 12

C) [1, 2, 6, 24]

D) 10

[Day 05: RegEx Grouping and sub()]

Q138. What is the output of re.sub(r'(\d{2})-(\d{2})', r'\2/\1', '10-25')?

```
import re
res = re.sub(r&amp;quot;(\d{2})-(\d{2})&amp;quot;, r&amp;quot;\2/\1&amp;quot;, &amp;quot;10-25&amp;quot;)
print(res)
```

A) 10-25

B) \2/\1

C) 10/25

D) 25/10

[Day 05: Lambda Sorting by Multiple Keys]

Q139. What is the output of sorting students by grade ascending, then age descending?

```
students = [(&amp;quot;Bob&amp;quot;, 20, &amp;quot;B&amp;quot;), (&amp;quot;Alice&amp;quot;, 22, &amp;quot;A&amp;quot;), (&amp;quot;Charlie&amp;quot;, 19, &amp;quot;A&amp;quo
students.sort(key=lambda s: (s[2], -s[1]))
print([s[0] for s in students])
```

A) ['Alice', 'Bob', 'Charlie']

B) ['Alice', 'Charlie', 'Bob']

C) ['Charlie', 'Alice', 'Bob']

D) ['Bob', 'Alice', 'Charlie']

[Day 05: Unpacking in Function Calls]

Q140. What is the output of the following function call using * and ** unpacking?

```
def display(a, b, c=10):
    return a + b + c
args = (5,)
kwargs = {&amp;quot;b&amp;quot;: 15, &amp;quot;c&amp;quot;: 20}
print(display(*args, **kwargs))
```

A) 30

B) 40

C) 50

D) TypeError

[Day 05: Recursion Base Case]

Q141. What happens if a recursive function in Python has no terminating base case?

A) RecursionError: maximum recursion depth exceeded

B) It loops infinitely without raising any error

C) Returns None automatically

D) System crash with Segmentation Fault

[Day 05: RegEx re.split()]

Q142. What does re.split(r'[,;\s]+', 'apple, banana; orange grape') produce?

A) ['apple', '', 'banana', '', 'orange', 'grape']

B) ['apple']

C) ['apple', 'banana', 'orange', 'grape']

D) ['apple,', 'banana;', 'orange', 'grape']

[Day 05: Function Annotation Attributes]

Q143. Where are function parameter and return type annotations stored at runtime?

```
def greet(name: str) -&gt; str:
    return &quot;Hi &quot; + name
```

A) greet.__signature__

B) greet.__doc__

C) greet.__annotations__

D) greet.__types__

[Day 05: Basic Decorator Syntax]

Q144. What is the equivalent syntax for decorating a function with @my_decorator?

```
@my_decorator
def func(): pass
```

A) func = my_decorator(func)

B) func = my_decorator()

C) my_decorator = func(my_decorator)

D) my_decorator.call(func)

[Day 05: Scope LEGB Rule]

Q145. What is the resolution order Python follows when looking up an unqualified variable name?

A) Local -> Enclosing -> Global -> Built-in

B) Local -> Global -> Enclosing -> Built-in

C) Built-in -> Global -> Enclosing -> Local

D) Global -> Enclosing -> Local -> Built-in

[Day 06: Class Attribute Shadowing]

Q146. Consider the following class definition and attribute assignments: class Device: count = 0 def __init__(self, name): self.name = name Device.count += 1 d1 = Device('Router') d2 = Device('Switch') d1.count = 10 print(d1.count, d2.count, Device.count) What is the output?

```
class Device:
    count = 0
    def __init__(self, name):
        self.name = name
        Device.count += 1
d1 = Device(&quot;Router&quot;)
d2 = Device(&quot;Switch&quot;)
d1.count = 10
print(d1.count, d2.count, Device.count)
```

A) 2 2 2

B) 10 2 10

C) 10 10 10

D) 10 2 2

[Day 06: Property Setter with Validation]

Q147. Consider the following class using a property setter with validation: class Wallet: def __init__(self, balance): self.balance = balance @property def balance(self): return self.__balance @balance.setter def balance(self, amount): if amount < 0: raise ValueError('Negative balance') self.__balance = amount w = Wallet(50) try: w.balance -= 100 except ValueError: w.balance += 20 print(w.balance) What is the output?

```
class Wallet:
    def __init__(self, balance):
        self.balance = balance
    @property
    def balance(self):
        return self.__balance
    @balance.setter
    def balance(self, amount):
        if amount &amp;lt; 0:
            raise ValueError(&amp;quot;Negative balance&amp;quot;)
        self.__balance = amount
w = Wallet(50)
try:
    w.balance -= 100
except ValueError:
    w.balance += 20
print(w.balance)
```

A) -50

B) 70

C) -30

D) 20

[Day 06: Name Mangling for Private Attributes]

Q148. How does Python mangle a private attribute named '__secret' inside class 'Account'?

```
class Account:
    def __init__(self):
        self.__secret = 1234
```

A) It makes it completely inaccessible outside the class

B) It rewrites it internally as _Account__secret

C) It encrypts the variable name using MD5

D) It moves it to __dict__['secret']

[Day 06: @classmethod vs @staticmethod]

Q149. What is the fundamental difference in parameter passing between @classmethod and @staticmethod?

A) @classmethod receives 'self', while @staticmethod receives 'cls'.

B) @staticmethod can modify class state, while @classmethod cannot.

C) @classmethod can only be called from an instance.

D) @classmethod receives the class object ('cls') as its first parameter; @staticmethod receives neither self nor cls.

[Day 06: Dunder __repr__ vs __str__]

Q150. When both __str__ and __repr__ are defined in a class, which one does repr() invoke, and which one does str() invoke?

A) repr() invokes __str__; str() invokes __repr__

B) Both invoke __repr__

C) Both invoke __str__

D) repr() invokes __repr__; str() invokes __str__ (falling back to __repr__ if __str__ is missing)

[Day 06: Dunder __len__ Protocol]

Q151. What requirement must the return value of __len__() fulfill?

A) It can return negative integers for reverse sequences

B) It can return any string or integer

C) It must return a non-negative integer (>= 0)

D) It must return a float

[Day 06: Operator Overloading (__add__)]

Q152. What is the output of the following Vector addition code?

```
class Vector:
    def __init__(self, x, y):
        self.x, self.y = x, y
    def __add__(self, other):
        return Vector(self.x + other.x, self.y + other.y)
v1 = Vector(1, 2)
v2 = Vector(3, 4)
v3 = v1 + v2
print(v3.x, v3.y)
```

A) 1 2

B) 3 4

C) 4 6

D) TypeError

[Day 06: super() with Single Inheritance]

Q153. What does super().__init__() do in a child class constructor?

```
class A:
    def __init__(self):
        self.val = 10
class B(A):
    def __init__(self):
        super().__init__()
        self.val += 5
b = B()
print(b.val)
```

A) 15

B) 5

C) AttributeError

D) 10

[Day 06: Dunder __eq__ and Hashability]

Q154. What happens to a class's default __hash__ when it implements custom __eq__ without defining __hash__?

```
class Point:
    def __init__(self, x):
        self.x = x
    def __eq__(self, other):
        return self.x == other.x
```

A) Python automatically hashes all instance attributes

B) Point objects can still be used as set elements

C) __hash__ remains the default object memory hash

D) Python automatically sets __hash__ = None, making instances unhashable

[Day 06: Polymorphism and Method Overriding]

Q155. What is the output of the following polymorphic function call?

```
class Dog:
    def speak(self): return &amp;quot;Woof&amp;quot;
class Cat:
    def speak(self): return &amp;quot;Meow&amp;quot;
def animal_sound(sounder):
    return sounder.speak()
print([animal_sound(a) for a in [Dog(), Cat()]])
```

A) TypeError

B) ['Woof', 'Meow']

C) ['Woof', 'Woof']

D) ['Meow', 'Meow']

[Day 07: File seek() and tell()]

Q156. Suppose a text file 'test.txt' contains the 10-character string 'abcdefghij'. What is the output of the following code?

```
with open(&amp;quot;test.txt&amp;quot;, &amp;quot;r&amp;quot;, encoding=&amp;quot;utf-8&amp;quot;) as f:
    f.seek(4)
    chunk = f.read(3)
    pos = f.tell()
    print(chunk, pos)
```

A) cde 7

B) def 6

C) efg 7

D) efg 6

[Day 07: Pickle File Mode Requirement]

Q157. When using the pickle module to serialize a Python object directly to a file on disk, in which access mode must the file be opened?

A) 'r'

B) 'wb'

C) 'a'

D) 'w'

[Day 07: csv.DictReader Structure]

Q158. What does each row produced by csv.DictReader represent?

A) A dictionary mapping header column names to row string values

B) A JSON object string

C) A tuple of values

D) A list of comma-separated strings

[Day 07: json.loads vs json.load]

Q159. What is the difference between json.loads() and json.load()?

A) They are aliases for each other.

B) json.loads() parses from a string; json.load() parses from a file-like stream object.

C) json.load() parses from a string; json.loads() parses from a file.

D) json.loads() is for lists only; json.load() is for dicts.

[Day 07: SQLite Parameterized Queries]

Q160. Why should SQL queries with variables always use parameterized placeholders (e.g., '?') instead of string formatting?

```
cursor.execute(&amp;quot;SELECT * FROM users WHERE name = ?&amp;quot;, (user_input,))
```

A) String formatting is deprecated in Python 3

B) To prevent SQL Injection attacks by letting the DB driver properly escape parameters

C) Parameterized queries run 100x faster

D) SQLite does not support string concatenation

[Day 07: SQLite commit() Requirement]

Q161. In Python's sqlite3 module, what happens to INSERT and UPDATE queries if conn.commit() is NOT called before closing the connection?

A) Changes are rolled back and not persisted to disk

B) An OperationalError is raised

C) The database file gets corrupted

D) Changes are committed automatically on close

[Day 07: File 'r+' Mode]

Q162. What does opening a file with mode 'r+' allow you to do?

A) Write only, creating the file if missing

B) Append only to binary files

C) Read only, but with root permissions

D) Read and write without truncating existing file contents

[Day 07: File 'w' Truncation]

Q163. What happens to an existing file when it is opened with mode 'w'?

A) It opens in read-only mode

B) It appends to the existing file

C) Its existing contents are immediately truncated (erased to 0 bytes)

D) A FileExistsError is raised

[Day 07: File readline() Trailing Newline]

Q164. What does f.readline() return when it reads a line from a text file, and how does it signal End of File (EOF)?

A) Returns a string including the trailing '\n'; returns '' (empty string) at EOF

B) Strips the newline automatically; returns None at EOF

C) Returns a list of characters; raises EOFError at EOF

D) Returns a byte array; returns -1 at EOF

[Day 07: SQLite fetchall() Return Type]

Q165. What does cursor.fetchall() return after executing a SELECT query in sqlite3?

```
cursor.execute(&amp;quot;SELECT id, name FROM users&amp;quot;)
rows = cursor.fetchall()
```

A) A list of tuples, where each tuple represents a row

B) A pandas DataFrame

C) A JSON string

D) A dictionary of rows

[Day 08: Generator Function Invocation]

Q166. In Python, what occurs when a generator function containing the yield statement is called?

```
def my_gen():
    print(&amp;quot;Starting&amp;quot;)
    yield 1
g = my_gen()
```

A) It executes up to the first yield statement and prints 'Starting'

B) It raises a SyntaxError if it does not also include a return statement

C) It executes the function body to completion and returns a tuple of results

D) It immediately returns a generator iterator object without executing the function body

[Day 08: Generator next() and StopIteration]

Q167. What is the output of the following generator code?

```
def count_up():
    yield 1
    yield 2
c = count_up()
print(next(c), next(c))
try:
    next(c)
except StopIteration:
    print(&amp;quot;Done&amp;quot;)
```

A) 1 1\nDone

B) 1 2\nDone

C) 1 2\nNone

D) 1 2\nStopIteration

[Day 08: Custom Iterator Protocol]

Q168. Which two special methods must an object implement to satisfy the Python Iterator Protocol?

A) __init__ and __del__

B) __iter__ and __next__

C) __enter__ and __exit__

D) __len__ and __getitem__

[Day 08: Generator Expression Memory Efficiency]

Q169. What is the primary architectural advantage of a generator expression over a list comprehension?

```
gen = (x * 2 for x in range(1_000_000))
lst = [x * 2 for x in range(1_000_000)]
```

A) Generator expressions evaluate lazily one item at a time, using O(1) constant memory instead of allocating a large list in RAM.

B) Generator expressions can be indexed with square brackets gen[0].

C) Generator expressions can be iterated over multiple times.

D) Generator expressions execute twice as fast.

[Day 08: itertools.chain()]

Q170. What is the output of list(itertools.chain([1, 2], ['a', 'b']))?

```
import itertools
res = list(itertools.chain([1, 2], [&amp;quot;a&amp;quot;, &amp;quot;b&amp;quot;]))
print(res)
```

A) [(1, 'a'), (2, 'b')]

B) {'1': 'a', '2': 'b'}

C) [[1, 2], ['a', 'b']]

D) [1, 2, 'a', 'b']

[Day 08: yield from Syntax]

Q171. What does 'yield from sub_iterable' do inside a generator function?

```
def combo():
    yield from [10, 20]
    yield 30
```

A) Terminates the generator

B) Yields the list [10, 20] as a single entity

C) Delegates iteration to sub_iterable, yielding each of its items transparently

D) Imports a generator from another file

[Day 08: Built-in iter() with Sentinel]

Q172. What does the two-argument form iter(callable, sentinel) do?

A) Iterates over sentinel using callable

B) Repeatedly calls callable with no arguments until it returns sentinel, then stops

C) Raises TypeError because iter takes 1 argument

D) Creates a circular iterator

[Day 08: Generator State Persistence]

Q173. What is the output of the following generator function?

```
def tracker():
    count = 0
    while True:
        count += 1
        yield count
t = tracker()
print(next(t), next(t), next(t))
```

A) 1 2 3

B) 0 1 2

C) Infinite loop

D) 1 1 1

[Day 08: Exception Handling (try-except-else-finally)]

Q174. Consider the following function with try-except-finally return values: def calculate_status(val): try: if val < 0: raise ValueError('Invalid number') return 'PROCESSED' except ValueError: return 'CAUGHT_ERROR' finally: return 'CLEANUP_DONE' print(calculate_status(-5), calculate_status(10)) What is the output?

```
def calculate_status(val):
    try:
        if val &amp;lt; 0:
            raise ValueError(&amp;quot;Invalid number&amp;quot;)
        return &amp;quot;PROCESSED&amp;quot;
    except ValueError:
        return &amp;quot;CAUGHT_ERROR&amp;quot;
    finally:
        return &amp;quot;CLEANUP_DONE&amp;quot;
print(calculate_status(-5), calculate_status(10))
```

A) CAUGHT_ERROR CLEANUP_DONE

B) CLEANUP_DONE PROCESSED

C) CAUGHT_ERROR PROCESSED

D) CLEANUP_DONE CLEANUP_DONE

[Day 08: Try-Except-Else Block]

Q175. When does the else block of a try-except statement execute?

```
try:
    x = 10 / 2
except ZeroDivisionError:
    print(&amp;quot;Error&amp;quot;)
else:
    print(&amp;quot;Success&amp;quot;)
finally:
    print(&amp;quot;Done&amp;quot;)
```

A) Never

B) Only if an exception was raised and caught

C) Only if NO exception was raised in the try block

D) Always, before the finally block

[Day 09: Flask Route Dynamic Parameters]

Q176. In Flask, how do you define a route variable rule that accepts only integers as book_id?

A) @app.route('/books/<int:book_id>')

B) @app.route('/books/<book_id:int>')

C) @app.route('/books/{int book_id}')

D) @app.route('/books/%d')

[Day 09: Flask HTTP Methods Configuration]

Q177. By default, which HTTP methods does a Flask route handler accept if the methods argument is omitted?

```
@app.route(&amp;quot;/items&amp;quot;)
def get_items(): pass
```

A) GET only

B) ALL HTTP methods

C) POST only

D) GET and POST

[Day 09: MVC vs MVT Pattern]

Q178. In Django's Model-View-Template (MVT) architecture, which component corresponds to the Controller in the classic MVC design pattern?

A) Template (templates/*.html)

B) Dispatcher (urls.py)

C) Model (models.py)

D) View (views.py)

[Day 10: BeautifulSoup Scraping Tag & Attribute]

Q179. Consider the following BeautifulSoup web scraping code: from bs4 import BeautifulSoup html = 'Click Here' soup = BeautifulSoup(html, 'html.parser') tag = soup.find('a') print(tag.text, tag['href']) What is the output?

```
from bs4 import BeautifulSoup
html = &#x27;&lt;a href=&quot;https://example.com&quot; class=&quot;link-btn&quot;&gt;Click Here&lt;/a&gt;&#x27;
soup = BeautifulSoup(html, &quot;html.parser&quot;)
tag = soup.find(&quot;a&quot;)
print(tag.text, tag[&quot;href&quot;])
```

A) link-btn https://example.com

B) Click Here https://example.com

C) Click Here link-btn

D) <a href="https://example.com"> Click Here

[Day 10: REST API Status Code 204]

Q180. What does HTTP status code 204 No Content indicate in a REST API?

A) The requested resource could not be found

B) The server successfully processed the request, but is not returning any content in the response body (common for DELETE operations)

C) The client sent an empty request body

D) The server encountered an internal error

[Day 10: HTTP Verbs Idempotency]

Q181. Which of the following HTTP methods is NOT idempotent according to REST conventions?

A) PUT

B) DELETE

C) POST

D) GET

[Day 10: Flask request.get_json()]

Q182. In a Flask REST API view, what does request.get_json() return when incoming request has 'Content-Type: application/json'?

```
@app.route(&quot;/api/users&quot;, methods=[&quot;POST&quot;])
def add_user():
    data = request.get_json()
```

A) A Werkzeug Response object

B) An XML tree

C) A raw JSON string

D) A parsed Python dictionary or list

[Day 10: BeautifulSoup find_all()]

Q183. What does soup.find_all('p', class_='intro') return?

A) A string containing concatenated paragraphs

B) The first matching <p> tag

C) A list of all matching <p> tag elements with class 'intro'

D) A dictionary of text contents

[Day 10: Flask jsonify() Helper]

Q184. What are the two primary things Flask's jsonify({'key': 'val'}) does?

A) Serializes data to JSON and sets Content-Type header to 'application/json'

B) Compresses JSON with gzip and saves to disk

C) Validates JSON schema against SQLite database

D) Converts dict to HTML table

[Day 10: HTTP 405 Status Code]

Q185. When does Flask return HTTP status code 405 Method Not Allowed?

A) When a client attempts an HTTP method (e.g. POST) not registered in the route's methods list

B) When authentication credentials are missing

C) When server encounters an uncaught exception

D) When an invalid URL route is requested

[Day 10: NumPy 2D Array Slicing]

Q186. Given arr = np.array([[10, 20, 30], [40, 50, 60], [70, 80, 90]]), what is arr[:2, 1:]?

```
import numpy as np
arr = np.array([[10, 20, 30], [40, 50, 60], [70, 80, 90]])
sub = arr[:2, 1:]
print(sub)
```

A) [[50, 60], [80, 90]]

B) [[20, 50], [30, 60]]

C) [[20, 30], [50, 60]]

D) [[10, 20], [40, 50]]

[Day 10: NumPy reshape() with -1]

Q187. What is the resulting shape when arr = np.arange(12).reshape((3, -1)) is executed?

```
import numpy as np
arr = np.arange(12).reshape((3, -1))
print(arr.shape)
```

A) ValueError

B) (3, 4)

C) (4, 3)

D) (3, 12)

[Day 11: Pandas Boolean Filtering Syntax]

Q188. In Pandas, why does executing df[df['Units'] > 2 and df['Price'] < 20] raise a ValueError, and what is the required syntax?

A) Column names must be prefixed with index; syntax must use df.query_filter()

B) Python's 'and' operator evaluates the truth value of the entire Series as ambiguous; the bitwise operator '&' with parentheses df[(df['Units'] > 2) & (df['Price'] < 20)] must be used

C) Pandas does not support multi-condition filtering on numerical columns

D) The operator '&&' must be used instead of 'and' without parentheses

[Day 11: Pandas .loc vs .iloc]

Q189. What is the key difference between df.loc[] and df.iloc[] in Pandas?

A) .loc is label-based (using row/column names); .iloc is integer-position based (0-indexed integers)

B) .loc is for rows only; .iloc is for columns only

C) They are completely identical

D) .loc is integer-position based; .iloc is label based

[Day 11: Pandas groupby() Aggregation]

Q190. Consider the following Pandas DataFrame aggregation: import pandas as pd df = pd.DataFrame({ 'Company': ['Lego', 'Lego', 'Duplo', 'Duplo', 'Lego'], 'Price': [10.0, 20.0, 15.0, 25.0, 30.0], 'Units': [2, 1, 3, 2, 1] }) grouped = df.groupby('Company').agg({'Price': ['mean', 'max'], 'Units': 'sum'}) print(grouped.shape, grouped['Price']['max']['Lego']) What is the output?

```
import pandas as pd
df = pd.DataFrame({
    &amp;quot;Company&amp;quot;: [&amp;quot;Lego&amp;quot;, &amp;quot;Lego&amp;quot;, &amp;quot;Duplo&amp;quot;, &amp;quot;Duplo&amp;quot;, &amp;quot;Lego&amp;quot;],
    &amp;quot;Price&amp;quot;: [10.0, 20.0, 15.0, 25.0, 30.0],
    &amp;quot;Units&amp;quot;: [2, 1, 3, 2, 1]
})
grouped = df.groupby(&amp;quot;Company&amp;quot;).agg({&amp;quot;Price&amp;quot;: [&amp;quot;mean&amp;quot;, &amp;quot;max&amp;quot;], &amp;quot;Units&amp;quot;: &amp;quot
print(grouped.shape, grouped[&amp;quot;Price&amp;quot;][&amp;quot;max&amp;quot;][&amp;quot;Lego&amp;quot;])
```

A) (5, 3) 20.0

B) (2, 3) 20.0

C) (2, 3) 30.0

D) (2, 2) 30.0

[Day 10: NumPy Boolean Masking]

Q191. What is the output of arr[arr % 2 != 0] on arr = np.array([1, 2, 3, 4, 5])?

A) 3

B) [2, 4]

C) [1, 3, 5]

D) [True, False, True, False, True]

[Day 11: Pandas Handling Missing Values]

Q192. What is the difference between df.dropna() and df.fillna(0)?

A) dropna() removes rows/columns with NaN values; fillna(0) replaces NaN values with 0.

B) dropna() replaces NaN with None; fillna(0) deletes NaN rows.

C) Both delete rows with NaN.

D) fillna(0) is only for string columns.

[Day 11: Pandas Series Creation]

Q193. What is the index of s = pd.Series([100, 200, 300]) if no index parameter was provided?

A) RangeIndex(start=0, stop=3, step=1) [0, 1, 2]

B) [1, 2, 3]

C) None

D) ['a', 'b', 'c']

[Day 11: Matplotlib subplots() Return Values]

Q194. What objects are returned by fig, ax = plt.subplots() in Matplotlib?

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots()
```

A) Two numpy arrays

B) A string filepath

C) A canvas and a brush

D) A Figure object (the overall window/page) and an Axes object (the actual plotting area)

[Day 11: Matplotlib plt.barh()]

Q195. What kind of chart does plt.barh() produce in Matplotlib?

A) Pie chart

B) Vertical bar chart

C) Horizontal bar chart

D) Histogram

[Day 10: NumPy Broadcasting Rules]

Q196. What is the result of adding a 1D array of shape (3,) to a 2D array of shape (2, 3)?

```
a = np.ones((2, 3))
b = np.array([1, 2, 3])
res = a + b
print(res.shape)
```

A) ValueError: shapes not aligned

B) (2, 3)

C) (3, 2)

D) (2, 1)

[Day 11: Pandas value_counts()]

Q197. What does df['Category'].value_counts() return?

A) A Series containing counts of unique values in 'Category', sorted in descending order

B) The mean value of 'Category'

C) A boolean Series

D) The total number of columns in df

[Day 10: Web Scraping robots.txt]

Q198. What is the standard purpose of the robots.txt file hosted on web servers?

A) To install Python scraping packages on client machines

B) To store encrypted passwords

C) To configure web server firewall rules

D) To inform web crawlers and scraping bots which URL paths are permitted or disallowed for scraping

[Day 10: Requests Library Response JSON]

Q199. What is the cleanest way to parse JSON data from a response using Python's requests library?

```
import requests
resp = requests.get(&amp;quot;https://api.example.com/data&amp;quot;)
```

A) resp.to_dict()

B) json.loads(resp.text)

C) resp.json()

D) Both A and B are valid, with resp.json() being the built-in convenience method

[Day 11: Matplotlib plt.savefig() Placement]

Q200. Why should plt.savefig('plot.png') typically be called BEFORE plt.show()?

A) plt.show() erases the disk file

B) plt.show() clears the current figure canvas when the GUI window is closed, resulting in a blank saved image if savefig is called after

C) savefig raises an error if called after show()

D) Matplotlib requires saving before display for licensing reasons

SECTION 3: ADVANCED LEVEL (Questions 201 - 300)

Expert Concepts: Python Internals, Memory Optimization (__slots__), MRO (C3 Linearization), super(), Metaclasses, Generators (.send/yield from), NumPy Strides & Pandas Performance

[Day 01: In-Place Modification inside Tuple]

Q201. Consider the following code executing in-place addition on a list inside a tuple: t = (1, [2, 3]) try: t[1] += [4] except TypeError: print('Error caught') print(t) What is the output?

```
t = (1, [2, 3])
try:
    t[1] += [4]
except TypeError:
    print(&amp;quot;Error caught&amp;quot;)
print(t)
```

A) Error caught\n(1, [2, 3, 4])

B) (1, [2, 3, 4])

C) TypeError: cannot catch this error

D) Error caught\n(1, [2, 3])

[Day 01: Walrus Operator (:=)]

Q202. What is the output of the following code utilizing the walrus operator (assignment expression)?

```
data = [1, 2, 3, 4, 5, 6]
res = [y for x in data if (y := x * 2) &amp;gt; 6]
print(res)
```

A) [7, 8, 9, 10, 11, 12]

B) SyntaxError

C) [8, 10, 12]

D) [4, 5, 6]

[Day 01: Structural Pattern Matching]

Q203. In Python 3.10+, what does the following match/case statement output?

```
command = (&amp;quot;move&amp;quot;, 10, 20)
match command:
    case (&amp;quot;quit&amp;quot;,):
        res = &amp;quot;Exiting&amp;quot;
    case (&amp;quot;move&amp;quot;, x, y):
        res = f&amp;quot;Moving to {x},{y}&amp;quot;
    case _:
        res = &amp;quot;Unknown&amp;quot;
print(res)
```

A) SyntaxError

B) Moving to 10,20

C) Unknown

D) Exiting

[Day 01: IEEE-754 Floating Point Precision]

Q204. What is the output of print(0.1 + 0.2 == 0.3, round(0.1 + 0.2, 2) == 0.3)?

```
print(0.1 + 0.2 == 0.3, round(0.1 + 0.2, 2) == 0.3)
```

A) True True

B) False True

C) True False

D) False False

[Day 01: Starred Expression Unpacking]

Q205. What is the output of the following extended iterable unpacking statement?

```
first, *middle, last = [10, 20, 30, 40, 50]
print(first, middle, last)
```

A) 10 (20, 30, 40) 50

B) [10] [20, 30, 40] [50]

C) ValueError

D) 10 [20, 30, 40] 50

[Day 02: Advanced String Formatting Specifier]

Q206. What is the output of the formatted string f'{1234567.89:*>15,._2f}'?

```
val = 1234567.89
print(f&amp;quot;{val:*&amp;gt;15,._2f}&amp;quot;)
```

A) **1,234,567.890

B) 1234567.890000*

C) ***1,234,567.89

D) 1,234,567.89***

[Day 02: String Interning Mechanics]

Q207. Under what circumstance does CPython automatically intern strings?

A) Strings containing spaces and emojis

B) All strings regardless of length or characters are always interned

C) Compile-time constants containing only ASCII letters, digits, and underscores (valid Python identifiers)

D) Only strings read dynamically from user input()

[Day 02: str.casefold() vs str.lower()]

Q208. Why is str.casefold() preferred over str.lower() when implementing caseless string matching?

A) casefold() runs 2x faster than lower()

B) casefold() implements aggressive Unicode full case-folding (e.g. converting German 'ß' to 'ss'), enabling true language-agnostic matching

C) casefold() strips punctuation as well

D) lower() cannot handle ASCII capital letters

[Day 01: Short-Circuit Side Effects]

Q209. What is the output of the following expression containing mutating side effects?

```
nums = []
def check():
    nums.append(1)
    return False
res = check() and check()
print(len(nums))
```

A) 1

B) 0

C) False

D) 2

[Day 02: bytes vs bytearray]

Q210. What is the key architectural difference between Python's bytes and bytearray types?

A) bytes stores Unicode, bytearray stores ASCII

B) Both are completely immutable

C) bytes is mutable, bytearray is immutable

D) bytes is immutable, bytearray is mutable

[Day 03: copy.deepcopy() Mechanics]

Q211. Consider the following deepcopy code with cyclical references: import copy a = [1, 2] a.append(a) b = copy.deepcopy(a) print(b[2] is b, b[2] is a) What is the output?

```
import copy
a = [1, 2]
a.append(a)
b = copy.deepcopy(a)
print(b[2] is b, b[2] is a)
```

A) RecursionError is raised

B) True False

C) True True

D) False False

[Day 04: Hashable Contract for Dictionary Keys]

Q212. What is the required contract for two objects in Python to be treated as identical keys in a dictionary?

A) They must have the same memory address (id(a) == id(b))

B) They must compare equal (a == b) and produce identical hash values (hash(a) == hash(b))

C) They must share the same class definition

D) They must both be strings

[Day 04: collections.defaultdict]

Q213. What is the output of the following code using collections.defaultdict?

```
from collections import defaultdict
d = defaultdict(list)
d[&amp;quot;fruits&amp;quot;].append(&amp;quot;apple&amp;quot;)
print(d[&amp;quot;vegetables&amp;quot;], d[&amp;quot;fruits&amp;quot;])
```

A) [] ['apple']

B) None ['apple']

C) [None] ['apple']

D) KeyError: 'vegetables'

[Day 04: collections.Counter Addition]

Q214. What is the output of adding two Counter objects?

```
from collections import Counter
c1 = Counter(a=3, b=1)
c2 = Counter(a=1, b=2, c=4)
res = c1 + c2
print(res[&amp;quot;a&amp;quot;], res[&amp;quot;b&amp;quot;], res[&amp;quot;c&amp;quot;])
```

A) 3 2 4

B) KeyError: 'c'

C) 3 1 4

D) 4 3 4

[Day 03: List Comprehension Scope in Python 3]

Q215. What is the output of the following code regarding comprehension variable leakage?

```
x = 100
res = [x for x in range(3)]
print(x)
```

A) NameError

B) 2

C) 100

D) 3

[Day 04: Custom Class as Dict Key]

Q216. What occurs when Point objects are used as dict keys in the following code?

```
class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y
    def __hash__(self):
        return hash((self.x, self.y))
    def __eq__(self, other):
        return (self.x, self.y) == (other.x, other.y)
d = {Point(1, 2): &amp;quot;Origin&amp;quot;}
print(d.get(Point(1, 2)))
```

A) None

B) Origin

C) TypeError: unhashable type: 'Point'

D) KeyError

[Day 04: Set Hash Collisions]

Q217. What is the output of len({1, True, 1.0}) in Python?

```
s = {1, True, 1.0}
print(len(s))
```

A) 1

B) 2

C) TypeError

D) 3

[Day 03: collections.namedtuple]

Q218. What is the primary characteristic of collections.namedtuple instances?

```
from collections import namedtuple
Task = namedtuple(&amp;quot;Task&amp;quot;, [&amp;quot;id&amp;quot;, &amp;quot;title&amp;quot;])
t = Task(1, &amp;quot;Fix bug&amp;quot;)
```

A) They can only store string values

B) They do not support indexing by integer position

C) They are mutable like dictionaries

D) They are immutable tuples with named field access and zero per-instance dict memory overhead

[Day 03: collections.deque Performance]

Q219. Why is collections.deque significantly faster than a Python list for implementing queues?

A) deque is implemented in assembly

B) deque compresses elements using gzip

C) deque provides O(1) amortized append and pop operations from both ends, whereas list.pop(0) and list.insert(0, v) are O(n)

D) deque uses multi-threading automatically

[Day 04: Dictionary Keys Union in Python 3]

Q220. What is the output of the set operation on dictionary keys?

```
d1 = {&amp;quot;a&amp;quot;: 1, &amp;quot;b&amp;quot;: 2}
d2 = {&amp;quot;b&amp;quot;: 3, &amp;quot;c&amp;quot;: 4}
print(d1.keys() &amp;amp; d2.keys())
```

A) {'b'}

B) [('b', 2), ('b', 3)]

C) {'b': 2}

D) TypeError: unhashable type: 'dict_keys'

[Day 05: Closure Late Binding Trap]

Q221. Consider the following code creating a list of lambda closures: multipliers = [lambda x: x * i for i in range(4)] results = [f(2) for f in multipliers] print(results) What is the output?

```
multipliers = [lambda x: x * i for i in range(4)]
results = [f(2) for f in multipliers]
print(results)
```

A) [6, 6, 6, 6]

B) [0, 1, 2, 3]

C) [8, 8, 8, 8]

D) [0, 2, 4, 6]

[Day 05: Fixing Closure Late Binding]

Q222. How do you correctly fix late binding in the closure loop so that each lambda captures the loop's current index value?

A) [lambda x: x * global(i) for i in range(4)]

B) [lambda x: x * nonlocal(i) for i in range(4)]

C) [lambda x: x * copy(i) for i in range(4)]

D) [lambda x, i=i: x * i for i in range(4)]

[Day 05: functools.wraps Role]

Q223. Why is @functools.wraps(fn) used inside custom function decorators?

```
from functools import wraps
def my_dec(f):
    @wraps(f)
    def wrapper(*args, **kwargs):
        return f(*args, **kwargs)
    return wrapper
```

A) To enforce type validation on arguments

B) To make the decorated function run asynchronously

C) To copy over original metadata such as __name__, __doc__, and __annotations__ to wrapper

D) To prevent the decorator from executing more than once

[Day 05: Decorator with Arguments (Decorator Factory)]

Q224. How many levels of nested functions are required to implement a decorator that accepts parameters (e.g. @repeat(num_times=3))?

A) 1 level

B) 2 levels

C) 3 levels

D) 4 levels

[Day 05: Multiple Decorator Stacking Order]

Q225. What is the execution order of stacked decorators? @dec1 @dec2 def func(): pass

```
@dec1
@dec2
def func(): pass
```

A) Applied from bottom to top: func = dec1(dec2(func)); during invocation, dec1's wrapper runs before dec2's wrapper

B) Executed randomly based on hash seed

C) Raises a SyntaxError

D) dec1 wraps func first, then dec2 wraps dec1

[Day 05: functools.lru_cache]

Q226. What does the @functools.lru_cache decorator do when applied to a recursive function like fibonacci(n)?

```
from functools import lru_cache
@lru_cache(maxsize=128)
def fib(n):
    if n &amp;lt; 2: return n
    return fib(n-1) + fib(n-2)
```

A) Runs the function in parallel across 128 CPU cores

B) Clears RAM when memory exceeds 128 MB

C) Limits recursion depth to 128 calls

D) Memoizes function return values based on input arguments, turning an O(2^n) exponential function into O(n) linear time

[Day 05: RegEx Lookaround Assertions]

Q227. What does the regular expression r'\d+(?=\s*USD)' match in 'Cost is 450 USD and 300 EUR'?

```
import re
text = &amp;quot;Cost is 450 USD and 300 EUR&amp;quot;
matches = re.findall(r&amp;quot;\d+(?=\s*USD)&amp;quot;, text)
print(matches)
```

A) []

B) ['450 USD']

C) ['450']

D) ['450', '300']

[Day 05: RegEx Negative Lookbehind]

Q228. What is matched by r'(?<!\$)\b\d+\b' in '$100 and 200'?

```
import re
text = &amp;quot;$100 and 200&amp;quot;
matches = re.findall(r&amp;quot;(?&amp;lt;!\$)\b\d+\b&amp;quot;, text)
print(matches)
```

A) ['100']

B) ['200']

C) ['100', '200']

D) ['$100', '200']

[Day 05: RegEx Named Capturing Groups]

Q229. How do you extract a named group 'year' from a regex match object?

```
import re
m = re.match(r&amp;quot;(?P&amp;lt;year&amp;gt;\d{4})-(?P&amp;lt;month&amp;gt;\d{2})&amp;quot;, &amp;quot;2026-10&amp;quot;)
print(m.group(&amp;quot;year&amp;quot;))
```

A) m['year']

B) m.year

C) m.named('year')

D) m.group('year')

[Day 05: Modifying Global vs UnboundLocalError]

Q230. What is the result of running the following code?

```
count = 10
def increment():
    count += 1
increment()
```

A) NameError: count is undefined

B) count remains 10

C) count becomes 11

D) UnboundLocalError: cannot access local variable 'count' where it is not associated with a value

[Day 06: MRO & Diamond Inheritance]

Q231. Consider the following classic diamond inheritance hierarchy: class A: def ping(self): return 'A' class B(A): def ping(self): return 'B' class C(A): def ping(self): return 'C' class D(B, C): pass What does D().ping() return?

```
class A:
    def ping(self): return &amp;quot;A&amp;quot;
class B(A):
    def ping(self): return &amp;quot;B&amp;quot;
class C(A):
    def ping(self): return &amp;quot;C&amp;quot;
class D(B, C):
    pass
print(D().ping())
```

A) AttributeError

B) A

C) B

D) C

[Day 06: Cooperative super() in Diamond Hierarchy]

Q232. What is the output of D().process() in the following cooperative multiple inheritance code?

```
class A:
    def process(self):
        return [&amp;quot;A&amp;quot;]
class B(A):
    def process(self):
        return [&amp;quot;B&amp;quot;] + super().process()
class C(A):
    def process(self):
        return [&amp;quot;C&amp;quot;] + super().process()
class D(B, C):
    def process(self):
        return [&amp;quot;D&amp;quot;] + super().process()
print(D().process())
```

A) ['D', 'B', 'A']

B) ['D', 'B', 'C', 'A']

C) ['D', 'B', 'A', 'C', 'A']

D) ['D', 'C', 'B', 'A']

[Day 06: __new__ vs __init__ (Singleton Pattern)]

Q233. What is the fundamental difference between __new__ and __init__?

A) __new__ is a static method responsible for allocating and returning a new instance; __init__ is called on that newly returned instance to initialize state

B) __new__ is only used for classes without inheritance

C) They are completely interchangeable

D) __init__ creates the instance; __new__ initializes it

[Day 06: Memory Optimization with __slots__]

Q234. What happens when a class defines __slots__ = ('name', 'age')?

```
class Person:
    __slots__ = (&amp;quot;name&amp;quot;, &amp;quot;age&amp;quot;)
```

A) The class can only be instantiated twice

B) The class can only inherit from tuple

C) The class suppresses the creation of an instance __dict__, saving substantial memory and restricting instance attributes strictly to 'name' and 'age'

D) The class attributes become read-only constants

[Day 06: Abstract Base Classes (abc.ABC)]

Q235. What occurs if a subclass inherits from an ABC containing an @abstractmethod but fails to implement that method?

```
from abc import ABC, abstractmethod
class Shape(ABC):
    @abstractmethod
    def area(self): pass
class Square(Shape):
    pass
s = Square()
```

A) s becomes None

B) Square instantiates normally, but calling area() raises NotImplementedError

C) TypeError: Can't instantiate abstract class Square without an implementation for abstract method 'area'

D) SyntaxError at class definition time

[Day 06: Dunder __call__ Method]

Q236. What does defining __call__ on a class allow instances of that class to do?

```
class Multiplier:
    def __init__(self, factor):
        self.factor = factor
    def __call__(self, val):
        return self.factor * val
m = Multiplier(5)
print(m(10))
```

A) It allows instances to be invoked like regular functions: m(10)

B) It automatically calls the garbage collector

C) It allows instances to be used as list indices

D) It converts the object to a string

[Day 06: Descriptor Protocol]

Q237. Which three dunder methods constitute the Python Descriptor Protocol?

A) __get__, __set__, __delete__

B) __iter__, __next__, __len__

C) __enter__, __exit__, __call__

D) __init__, __new__, __del__

[Day 06: Metaclass Definition]

Q238. In Python, what is a metaclass?

A) A module containing only abstract methods

B) The base class for all C-extensions

C) A class that cannot be inherited

D) A class whose instances are themselves classes (a class that defines how a class behaves)

[Day 06: Context Manager Protocol (__enter__ and __exit__)]

Q239. What must the __exit__(self, exc_type, exc_val, exc_tb) method return to suppress an exception raised inside the 'with' block?

A) A string containing the error message

B) None

C) True

D) False

[Day 06: Dunder __getitem__ and Iteration]

Q240. What happens if an object implements __getitem__ and __len__ but NOT __iter__ when passed to a 'for' loop?

```
class CustomSeq:
    def __len__(self): return 3
    def __getitem__(self, idx):
        if idx &gt;= 3: raise IndexError
        return idx * 10
print(list(CustomSeq()))
```

A) TypeError: 'CustomSeq' object is not iterable

B) Python falls back to old-style sequence iteration, calling __getitem__ starting at index 0 until IndexError: [0, 10, 20]

C) An infinite loop occurs

D) Raises AttributeError

[Day 07: File seek() with os.SEEK_END]

Q241. What does f.seek(-5, os.SEEK_END) do on a file opened in 'rb' mode?

A) Positions the file cursor 5 bytes before the end of the file

B) Deletes the last 5 bytes of the file

C) Raises an OSError in binary mode

D) Moves the cursor 5 bytes before the start of the file

[Day 07: Custom JSONEncoder]

Q242. How do you serialize a Python datetime object to JSON using json.dumps()?

A) datetime objects are natively JSON serializable by default

B) json.dumps(dt, format='%Y-%m-%d')

C) json.dumps(dt, enable_dates=True)

D) Provide a custom serializer via the 'default' parameter: json.dumps(dt, default=str)

[Day 07: Pickle Security Vulnerability]

Q243. Why is unpickling data received from an untrusted client or network source considered a critical security vulnerability?

A) Pickle exposes database passwords

B) It may contain malformed characters that crash the python parser

C) Pickle bytecode can execute arbitrary Python/system commands during deserialization via the __reduce__ method

D) Pickle files corrupt the local hard drive

[Day 07: SQLite sqlite3.Row Factory]

Q244. What is the benefit of setting conn.row_factory = sqlite3.Row in Python's sqlite3 module?

```
conn.row_factory = sqlite3.Row
cursor = conn.cursor()
row = cursor.execute(&amp;quot;SELECT id, name FROM users&amp;quot;).fetchone()
```

A) Enables accessing returned row columns by name as well as index (e.g. row['name'] and row[1])

B) Encrypts database rows on disk

C) Makes queries run in separate threads

D) Automatically converts rows to pandas DataFrame

[Day 07: SQLite Connection as Context Manager]

Q245. When an active sqlite3 connection is used as a context manager 'with conn:', what happens upon exiting the block?

```
with conn:
    conn.execute(&amp;quot;INSERT INTO logs VALUES (?)&amp;quot;, (&amp;quot;Event&amp;quot;,))
```

A) The transaction is committed automatically if no exceptions occurred, or rolled back if an exception was raised (the connection remains OPEN)

B) The database file is deleted

C) All tables are locked indefinitely

D) The database connection is closed automatically

[Day 07: In-Memory SQLite Database]

Q246. How do you create a fast, volatile SQLite database entirely in RAM in Python?

A) sqlite3.connect(ram=True)

B) sqlite3.in_memory()

C) sqlite3.connect('temp.db')

D) sqlite3.connect(':memory:')

[Day 07: contextlib.@contextmanager]

Q247. What does a generator function decorated with @contextmanager yield to the 'as' target of a 'with' statement?

```
from contextlib import contextmanager
@contextmanager
def open_resource():
    res = &amp;quot;Active&amp;quot;
    try:
        yield res
    finally:
        print(&amp;quot;Closed&amp;quot;)
```

A) None

B) The generator iterator object itself

C) The single value produced by the yield statement (res)

D) A tuple of all variables in scope

[Day 07: io.StringIO vs io.BytesIO]

Q248. What is the difference between io.StringIO and io.BytesIO in Python?

A) io.StringIO operates as an in-memory stream for text (str); io.BytesIO operates as an in-memory stream for binary data (bytes)

B) io.StringIO writes to disk, io.BytesIO writes to network

C) Both only handle JSON data

D) They are deprecated aliases for open()

[Day 07: csv.writer lineterminator Parameter]

Q249. On Windows systems, why is it critical to pass newline='' when opening a CSV file with open() for csv.writer?

```
with open(&amp;quot;data.csv&amp;quot;, &amp;quot;w&amp;quot;, newline=&amp;quot;&amp;quot;, encoding=&amp;quot;utf-8&amp;quot;) as f:
    w = csv.writer(f)
```

A) Windows throws a PermissionError without newline=''

B) To allow binary data inside CSV

C) To enforce UTF-16 encoding

D) To prevent csv.writer from producing double carriage returns (\r\r\n) causing extra blank rows

[Day 07: SQLite Executemany Performance]

Q250. Why is cursor.executemany(sql, seq_of_params) preferred over looping cursor.execute(sql, params) in a Python for-loop?

A) It disables database constraints

B) It converts SQLite into PostgreSQL

C) It skips data validation

D) It batches parameter execution, reducing query compilation overhead and context switching across the Python/C DB-API boundary

[Day 08: Bidirectional Generator .send()]

Q251. What is the output of the following generator utilizing the .send() method?

```
def accumulator():
    total = 0
    while True:
        val = yield total
        if val is None: break
        total += val
gen = accumulator()
next(gen)
print(gen.send(10), gen.send(20))
```

A) TypeError

B) 10 20

C) 10 30

D) 0 10

[Day 08: Priming a Generator Coroutine]

Q252. Why must next(gen) or gen.send(None) be called on a generator before sending any non-None value with gen.send(value)?

A) Python raises TypeError: can't send non-None value to a just-started generator

B) It clears the memory buffer

C) It establishes a TCP socket

D) It is an optional best practice that has no runtime effect

[Day 08: Generator .close() Method]

Q253. What exception is raised inside a suspended generator when .close() is called on it?

```
gen.close()
```

A) GeneratorExit

B) RuntimeError

C) SystemExit

D) StopIteration

[Day 08: yield from Return Value]

Q254. What is the value of 'res' in the following sub-generator delegation code?

```
def sub_gen():
    yield 1
    yield 2
    return &amp;quot;COMPLETED&amp;quot;
def main_gen():
    res = yield from sub_gen()
    yield res
print(list(main_gen()))
```

A) ['COMPLETED']

B) [1, 2, StopIteration]

C) [1, 2]

D) [1, 2, 'COMPLETED']

[Day 08: itertools.islice with Generators]

Q255. What is the output of taking a slice from an infinite generator using itertools.islice?

```
import itertools
def count_from(n):
    while True:
        yield n
        n += 2
sliced = list(itertools.islice(count_from(10), 3))
print(sliced)
```

A) Infinite loop

B) [10, 12, 14]

C) [10, 11, 12]

D) [12, 14, 16]

[Day 08: inspect.isgeneratorfunction]

Q256. How does the Python runtime determine if a function is a generator function?

A) By checking if it has a @generator decorator

B) By the presence of the 'yield' keyword anywhere in its compiled code object flags (CO_GENERATOR)

C) By checking its return type annotation

D) By checking if its name starts with 'gen_'

[Day 08: Custom Iterable Class]

Q257. What is the difference between an Iterable and an Iterator in Python?

A) They are identical concepts

B) Iterators can only be looped over once, while Iterables can never be looped over

C) An Iterable is always a generator

D) An Iterable implements __iter__ returning an iterator; an Iterator implements __next__ (and __iter__ returning self)

[Day 08: StopIteration Value Attribute]

Q258. When a generator function executes 'return 42', where does the value 42 go?

```
def my_gen():
    yield 1
    return 42
g = my_gen()
next(g)
try:
    next(g)
except StopIteration as e:
    print(e.value)
```

A) It is printed to stderr

B) Return statements with values are illegal inside generators

C) It is lost and discarded

D) It is stored in the StopIteration exception's .value attribute

[Day 08: Generator Expression Re-use]

Q259. What is the output of attempting to iterate over an already exhausted generator expression a second time?

```
gen = (x for x in [1, 2])
list1 = list(gen)
list2 = list(gen)
print(list1, list2)
```

A) StopIteration is raised

B) [1, 2] [1, 2]

C) [1, 2] []

D) [1, 2] [None]

[Day 08: Exception Handling Hierarchy]

Q260. Why does catching 'except Exception:' NOT intercept a generator's GeneratorExit or keyboard interrupts?

A) GeneratorExit and KeyboardInterrupt inherit directly from BaseException, not Exception

B) They are not Python exceptions

C) They can only be caught using try-finally

D) Python suppresses them automatically

[Day 09: Flask Application vs Request Context]

Q261. What is the difference between Flask's Application Context ('current_app', 'g') and Request Context ('request', 'session')?

A) Application context is for client side; Request context is for server side

B) Request context only exists for POST requests

C) There is no distinction in Flask

D) Application context is tied to the running application configuration and shared state; Request context tracks data specific to an individual incoming HTTP request

[Day 09: Flask Blueprints Purpose]

Q262. What is the primary architectural purpose of Flask Blueprints?

A) To manage database migrations

B) To compile Flask into C code

C) To generate CSS blueprints for front-end styling

D) To organize a large application into reusable, modular sub-components with their own URL prefixes, templates, and static directories

[Day 10: REST PUT vs PATCH Semantics]

Q263. According to standard REST conventions, what is the exact semantic difference between HTTP PUT and HTTP PATCH?

A) PATCH is idempotent; PUT is never idempotent

B) PUT and PATCH are completely identical

C) PUT replaces the entire target resource with the uploaded representation; PATCH applies partial modifications/deltas to the existing resource

D) PUT is for creation only; PATCH is for deletion

[Day 10: HTTP 401 vs 403 Status Codes]

Q264. What is the precise difference between HTTP 401 Unauthorized and HTTP 403 Forbidden?

A) 401 means Authentication is required or credentials failed; 403 means the user is authenticated, but lacks Authorization (permission) to access the resource

B) 401 is for server errors; 403 is for client errors

C) 401 is for GET requests; 403 is for POST requests

D) They are synonyms

[Day 10: Flask Custom Error Handler]

Q265. How do you register a custom 404 JSON error response in Flask?

A) @app.errorhandler(404) def not_found(e): return jsonify(error='Not Found'), 404

B) @app.route('/404')

C) @app.handle_error(404)

D) @app.catch(404)

[Day 10: BeautifulSoup CSS Selector select()]

Q266. In BeautifulSoup, what is returned by soup.select('div.card > a.btn[href^="https"]')?

A) A boolean indicating whether such elements exist

B) An iterator of strings

C) The first matching element as a Tag object

D) A list of all Tag objects matching the CSS selector (direct anchor children of div.card with class 'btn' starting with https)

[Day 10: Web Scraping User-Agent Header]

Q267. Why is it often necessary to supply a custom 'User-Agent' header when sending HTTP GET requests using Python's requests library?

```
headers = {&amp;quot;User-Agent&amp;quot;: &amp;quot;Mozilla/5.0 (Windows NT 10.0; Win64; x64)&amp;quot;}
resp = requests.get(url, headers=headers)
```

A) To compress the returned HTML response

B) requests will not compile without headers

C) Many web servers block or return HTTP 403 to requests with the default 'python-requests/x.x' User-Agent to prevent bot scraping

D) To enable SSL certificate validation

[Day 10: requests.Session() Connection Pooling]

Q268. What is the primary performance benefit of using a requests.Session() object when scraping multiple pages from the same host?

```
with requests.Session() as s:
    for url in urls:
        resp = s.get(url)
```

A) It automatically executes JavaScript

B) It persists cookies and reuses the underlying TCP connection (HTTP Keep-Alive), eliminating repeated TCP and TLS handshakes

C) It bypasses CAPTCHA challenges automatically

D) It multithreads the network socket

[Day 10: BeautifulSoup NavigableString vs Tag]

Q269. What is the difference between soup.find('p') and soup.find('p').string?

A) Both return plain Python strings

B) find('p').string returns an integer length

C) find('p') returns an XML parser

D) find('p') returns a Tag object; .string returns a NavigableString containing the text

[Day 10: REST Architectural Constraints]

Q270. Which of the following is NOT one of Roy Fielding's 6 foundational REST architectural constraints?

A) Cacheability

B) Mandatory Object-Relational Mapping (ORM)

C) Statelessness

D) Client-Server Separation

[Day 10: NumPy 3D Array Striding & Slicing]

Q271. Given arr = np.arange(24).reshape((2, 3, 4)), what is the shape and the element sub[0, 0, 0] of the slice sub = arr[:, ::-1, 1:3]?

```
import numpy as np
arr = np.arange(24).reshape((2, 3, 4))
sub = arr[:, ::-1, 1:3]
print(sub.shape, sub[0, 0, 0])
```

A) (2, 3, 3) and 8

B) (2, 3, 2) and 9

C) (2, 3, 2) and 1

D) (2, 2, 2) and 9

[Day 10: NumPy View vs Copy Slicing]

Q272. Unlike standard Python lists where slicing creates a shallow copy, what does basic slicing on a NumPy ndarray produce?

```
a = np.array([1, 2, 3])
b = a[1:]
b[0] = 99
print(a[1])
```

A) 2

B) 99

C) IndexError

D) ValueError

[Day 10: NumPy Fancy Indexing creates Copy]

Q273. What is the key difference between basic slicing and fancy indexing (integer array indexing) in NumPy?

```
arr = np.array([10, 20, 30, 40])
view = arr[1:3]
copy = arr[[1, 2]]
```

A) Both return views

B) Fancy indexing is only for 1D arrays

C) Basic slicing is slower than fancy indexing

D) Basic slicing returns a view; Fancy indexing always creates and returns a copy

[Day 11: Pandas SettingWithCopyWarning]

Q274. What common code pattern triggers Pandas' SettingWithCopyWarning?

```
df[df[&amp;quot;status&amp;quot;] == &amp;quot;active&amp;quot;][&amp;quot;discount&amp;quot;] = 0.1
```

A) Calling df.dropna() without parameters

B) Missing integer index

C) Using string columns instead of categories

D) Chained indexing assignment (e.g. df[cond]['col'] = val), because Pandas cannot guarantee whether df[cond] returned a view or a copy

[Day 11: Pandas MultiIndex Cross-Section (.xs)]

Q275. What does df.xs('Lego', level='Company') do on a DataFrame with a MultiIndex on rows?

A) Resets the entire index to integers

B) Deletes the 'Lego' company rows

C) Returns a cross-section of data for the 'Lego' level key, dropping that level from the index by default

D) Exports the rows to XML

[Day 11: Pandas Performance Hierarchy]

Q276. In what order of execution speed (fastest to slowest) do typical row-wise operations in Pandas rank?

A) Vectorized operations > df.apply() > df.itertuples() > df.iterrows()

B) df.iterrows() > df.itertuples() > df.apply() > Vectorized operations

C) df.apply() > Vectorized operations > df.iterrows() > df.itertuples()

D) All have identical performance in Python 3

[Day 11: Matplotlib Twin Axes (ax.twinx)]

Q277. In Matplotlib, what does ax2 = ax.twinx() accomplish?

```
fig, ax = plt.subplots()
ax2 = ax.twinx()
```

A) Creates a second independent y-axis sharing the exact same x-axis on the same plot (dual-axis chart)

B) Inverts the x-axis values

C) Copies the figure to the clipboard

D) Clones the plot side-by-side into two separate subplots

[Day 11: Pandas Categorical Data Type]

Q278. What is the primary benefit of converting a string column with few unique values to category dtype (df['City'].astype('category'))?

A) It prevents null values

B) It sorts the DataFrame automatically

C) It allows values to be printed in bold

D) Substantially reduced memory usage (storing integer codes instead of duplicate strings) and faster groupby computations

[Day 10: NumPy Axis Parameter in Aggregation]

Q279. What does np.sum(arr, axis=0) compute on a 2D array of shape (4, 5)?

```
arr = np.ones((4, 5))
res = np.sum(arr, axis=0)
print(res.shape)
```

A) Shape (4, 5)

B) Sums along rows, collapsing axis 0 to return 5 column sums (shape (5,))

C) Sums along columns, collapsing axis 1 to return 4 row sums (shape (4,))

D) Sums all 20 elements to a scalar

[Day 11: Matplotlib Object-Oriented API vs pyplot]

Q280. Why is Matplotlib's Object-Oriented API ('fig, ax = plt.subplots(); ax.plot(...)') strongly recommended over the state-based pyplot API ('plt.plot(...)')?

A) pyplot is deprecated and removed in recent versions

B) The Object-Oriented API provides explicit, isolated control over figures and axes without relying on implicit global state, avoiding cross-talk in multi-figure/multi-plot scripts

C) The OO API executes 10x faster

D) pyplot cannot draw bar charts

[Day 01: Global Interpreter Lock (GIL) Concept]

Q281. What is Python's Global Interpreter Lock (GIL) in standard CPython?

A) A mutex that prevents multiple native OS threads from executing Python bytecodes simultaneously in a single process

B) A security lock preventing unauthorized Python module imports

C) A hardware lock on the CPU core

D) A lock that prevents files from being modified by two programs

[Day 02: String intern() Explicit Invocation]

Q282. What is the output of sys.intern('hello ' + 'world') is sys.intern('hello world')?

```
import sys
s1 = sys.intern(&amp;quot;hello &amp;quot; + &amp;quot;world&amp;quot;)
s2 = sys.intern(&amp;quot;hello world&amp;quot;)
print(s1 is s2)
```

A) TypeError

B) None

C) True

D) False

[Day 03: Bisect Module for Sorted Sequences]

Q283. What does bisect.bisect_left([10, 20, 30, 40], 25) return?

```
import bisect
idx = bisect.bisect_left([10, 20, 30, 40], 25)
print(idx)
```

A) -1

B) 1

C) 2

D) 3

[Day 04: Dict Compact Layout in Python 3.6+]

Q284. How did Raymond Hettinger's compact dict implementation in Python 3.6+ reduce dictionary memory consumption by 20-25% while guaranteeing insertion order?

A) By compressing keys using gzip in RAM

B) By splitting the hash table into a sparse index array of byte indices and a compact dense entries array of (hash, key, value)

C) By converting keys to integers

D) By using binary search trees instead of hash tables

[Day 05: Function __closure__ Attribute]

Q285. Where does Python store free variables referenced by a closure function?

```
def make_adder(x):
    def adder(y):
        return x + y
    return adder
a = make_adder(10)
```

A) In a tuple of cell objects accessible via adder.__closure__

B) On the operating system stack

C) In sys.modules

D) In a global dictionary

[Day 06: Method Type Binding (Bound Method)]

Q286. What is the difference between Person.greet and p.greet where p = Person()?

```
class Person:
    def greet(self): pass
p = Person()
```

A) p.greet cannot be passed as a callback

B) Person.greet raises a TypeError when defined

C) Both are identical functions

D) Person.greet is a function; p.greet is a bound method that automatically curries 'p' as the first argument 'self'

[Day 06: Class Creation via type()]

Q287. How do you dynamically create a class named 'User' with attribute role='admin' using the type() metaclass constructor directly?

A) User = type('User', role='admin')

B) User = type('User', (), {'role': 'admin'})

C) User = type.create('User', {'role': 'admin'})

D) User = class('User', (), role='admin')

[Day 07: SQLite Foreign Key Constraints Enforcement]

Q288. Why does SQLite by default allow inserting rows that violate FOREIGN KEY constraints, and how is enforcement enabled?

A) Foreign keys are not supported in SQLite

B) For historical backward-compatibility, foreign key enforcement is disabled by default; it must be enabled per-connection via 'PRAGMA foreign_keys = ON;'

C) Foreign keys are only checked during commit()

D) Python's DB-API disables foreign keys

[Day 08: Coroutine Delegation Protocol]

Q289. What is the key difference between a Python generator and an async coroutine defined with 'async def'?

A) Async def cannot use loops

B) Generators run in separate OS threads

C) They are completely identical in implementation

D) Generators produce streams of data via yield; async coroutines model asynchronous cooperative tasks managed by an event loop via await

[Day 09: WSGI Specification Architecture]

Q290. What is the standard function signature of a WSGI-compliant application callable according to PEP 3333?

A) application(http_stream)

B) application(url, headers, body)

C) application(request, response)

D) application(environ, start_response)

[Day 10: NumPy Vectorization vs CPython Loop]

Q291. Why is np.sum(large_array) orders of magnitude faster than Python's built-in sum(large_array)?

A) Python's sum() has a memory leak

B) NumPy operates on contiguous blocks of homogenous C data types without type-checking or pointer chasing per element

C) NumPy disables GIL and runs on GPU by default

D) Built-in sum() uses recursion

[Day 11: Pandas merge() Types]

Q292. What is the output of pd.merge(df1, df2, on='id', how='left') when an id in df1 has no match in df2?

A) The row is dropped

B) The row is retained, and columns from df2 are filled with NaN

C) A KeyError is raised

D) Columns from df2 are filled with empty strings ''

[Day 11: Pandas Indexing with .query()]

Q293. What is the advantage of using df.query('Price > 20 and Units < 5') over standard bracket indexing?

A) It runs faster on very large DataFrames by evaluating expressions with NumExpr

B) It provides a cleaner and more readable syntax

C) All of the above

D) It allows using natural logical operators ('and', 'or') without requiring bitwise operators ('&', '|') or parentheses

[Day 10: NumPy newaxis Dimension Expansion]

Q294. What is the shape of arr[:, np.newaxis] where arr = np.array([1, 2, 3, 4])?

```
arr = np.array([1, 2, 3, 4])
expanded = arr[:, np.newaxis]
print(expanded.shape)
```

A) (1, 4)

B) (4, 4)

C) (4,)

D) (4, 1)

[Day 05: Argument Packing and Keyword Ordering]

Q295. Which of the following function signatures is syntactically legal in Python 3?

A) def fn(a=1, b, **c): pass

B) def fn(a, *b, c=1, **d): pass

C) def fn(**d, *b, a): pass

D) def fn(*b, a, **d, e): pass

[Day 06: Method Overriding and super() Delegation]

Q296. What is the output of the following inheritance snippet?

```
class Base:
    def __init__(self):
        self.val = 1
class Derived(Base):
    def __init__(self):
        self.val = 2
        super().__init__()
d = Derived()
print(d.val)
```

A) 2

B) 1

C) 3

D) None

[Day 07: Text vs Binary Mode Cursor Offsets]

Q297. In Python 3 text mode ('r'), why does f.seek(offset, 1) or f.seek(offset, 2) raise an io.UnsupportedOperation when offset is non-zero?

A) Operating systems prohibit seek in text files

B) Python text files do not have file descriptors

C) Seek is only available in write mode

D) Multi-byte Unicode encodings (like UTF-8) mean character offsets do not match byte offsets, making arbitrary relative byte seeking invalid in text streams

[Day 10: BeautifulSoup find vs select]

Q298. What is the output of soup.find('div', id='main') vs soup.select_one('#main')?

A) select_one is only for classes

B) find raises an error if id is missing

C) Both locate and return the first matching Tag element with id='main' (or None if not found)

D) find returns a list; select_one returns a string

[Day 11: Pandas pivot_table() Aggregation]

Q299. What is the primary difference between df.pivot() and df.pivot_table() in Pandas?

A) Both do not support aggregation

B) df.pivot() requires unique index/column combinations (raises ValueError on duplicates); df.pivot_table() aggregates duplicate combinations using aggfunc (default mean)

C) df.pivot_table() is deprecated

D) df.pivot() only works on numbers

[Day 11: Matplotlib plt.subplots figsize Units]

Q300. In Matplotlib, what unit of measurement is used for figsize=(width, height) in plt.subplots(figsize=(10, 6))?

```
fig, ax = plt.subplots(figsize=(10, 6))
```

A) Centimeters

B) Pixels

C) Inches

D) Millimeters

APPENDIX A: COMPLETE MASTER ANSWER KEY (Q1 - Q300)

Compact verification matrix across all 300 questions for quick evaluation and score tabulation.

Q1: A	Q31: C	Q61: C	Q91: C	Q121: A	Q151: C	Q181: C	Q211: B	Q241: A	Q271: B
Q2: D	Q32: B	Q62: A	Q92: B	Q122: D	Q152: C	Q182: D	Q212: B	Q242: D	Q272: B
Q3: C	Q33: B	Q63: B	Q93: D	Q123: C	Q153: A	Q183: C	Q213: A	Q243: C	Q273: D
Q4: B	Q34: D	Q64: C	Q94: D	Q124: B	Q154: D	Q184: A	Q214: D	Q244: A	Q274: D
Q5: B	Q35: C	Q65: A	Q95: C	Q125: A	Q155: B	Q185: A	Q215: C	Q245: A	Q275: C
Q6: A	Q36: B	Q66: A	Q96: B	Q126: C	Q156: C	Q186: C	Q216: B	Q246: D	Q276: A
Q7: C	Q37: A	Q67: C	Q97: A	Q127: C	Q157: B	Q187: B	Q217: A	Q247: C	Q277: A
Q8: B	Q38: C	Q68: B	Q98: D	Q128: B	Q158: A	Q188: B	Q218: D	Q248: A	Q278: D
Q9: A	Q39: C	Q69: A	Q99: C	Q129: A	Q159: B	Q189: A	Q219: C	Q249: D	Q279: B
Q10: D	Q40: B	Q70: C	Q100: C	Q130: C	Q160: B	Q190: C	Q220: A	Q250: D	Q280: B
Q11: C	Q41: A	Q71: C	Q101: A	Q131: C	Q161: A	Q191: C	Q221: A	Q251: C	Q281: A
Q12: B	Q42: D	Q72: A	Q102: C	Q132: B	Q162: D	Q192: A	Q222: D	Q252: A	Q282: C
Q13: B	Q43: C	Q73: A	Q103: D	Q133: A	Q163: C	Q193: A	Q223: C	Q253: A	Q283: C
Q14: D	Q44: B	Q74: A	Q104: B	Q134: A	Q164: A	Q194: D	Q224: C	Q254: D	Q284: B
Q15: C	Q45: A	Q75: C	Q105: D	Q135: C	Q165: A	Q195: C	Q225: A	Q255: B	Q285: A
Q16: A	Q46: D	Q76: C	Q106: C	Q136: B	Q166: D	Q196: B	Q226: D	Q256: B	Q286: D
Q17: A	Q47: D	Q77: A	Q107: D	Q137: A	Q167: B	Q197: A	Q227: C	Q257: D	Q287: B
Q18: D	Q48: B	Q78: A	Q108: A	Q138: D	Q168: B	Q198: D	Q228: B	Q258: D	Q288: B
Q19: D	Q49: A	Q79: C	Q109: D	Q139: B	Q169: A	Q199: D	Q229: D	Q259: C	Q289: D
Q20: B	Q50: D	Q80: B	Q110: C	Q140: B	Q170: D	Q200: B	Q230: D	Q260: A	Q290: D
Q21: A	Q51: D	Q81: A	Q111: C	Q141: A	Q171: C	Q201: A	Q231: C	Q261: D	Q291: B
Q22: D	Q52: B	Q82: D	Q112: B	Q142: C	Q172: B	Q202: C	Q232: B	Q262: D	Q292: B
Q23: C	Q53: A	Q83: B	Q113: A	Q143: C	Q173: A	Q203: B	Q233: A	Q263: C	Q293: C
Q24: B	Q54: A	Q84: B	Q114: D	Q144: A	Q174: D	Q204: B	Q234: C	Q264: A	Q294: D
Q25: A	Q55: B	Q85: D	Q115: C	Q145: A	Q175: C	Q205: D	Q235: C	Q265: A	Q295: B
Q26: D	Q56: B	Q86: D	Q116: A	Q146: D	Q176: A	Q206: C	Q236: A	Q266: D	Q296: B
Q27: C	Q57: A	Q87: C	Q117: A	Q147: B	Q177: A	Q207: C	Q237: A	Q267: C	Q297: D
Q28: A	Q58: C	Q88: A	Q118: D	Q148: B	Q178: D	Q208: B	Q238: D	Q268: B	Q298: C
Q29: A	Q59: B	Q89: A	Q119: C	Q149: D	Q179: B	Q209: A	Q239: C	Q269: D	Q299: B
Q30: D	Q60: A	Q90: A	Q120: B	Q150: D	Q180: B	Q210: D	Q240: B	Q270: B	Q300: C

APPENDIX B: DETAILED EXPLANATIONS & KEY TAKEAWAYS

Comprehensive rationales, mechanics, and references for all 300 questions grouped sequentially.

Q1. [A] (Day 01: Python History & Origins) - Guido van Rossum developed Python in the late 1980s and first released it in February 1991.

Q2. [D] (Day 01: Language Architecture) - CPython first compiles source code into intermediate bytecode (.pyc), which is then interpreted by the Python Virtual Machine (PVM).

Q3. [C] (Day 01: Output Formatting) - The sep parameter specifies the separator placed between positional arguments in print().

Q4. [B] (Day 01: Print Formatting) - The end parameter replaces the default newline (\n). Setting end='***' appends '***' without starting a new line.

Q5. [B] (Day 01: Input Handling) - input() always reads input from standard input as a string (str), requiring explicit casting (e.g., int()).

Q6. [A] (Day 01: Identifiers & Naming Rules) - Variable names cannot start with a digit, cannot contain hyphens (-), and cannot be reserved keywords ('class').

Q7. [C] (Day 01: Floor Division) - Floor division // rounds down towards negative infinity. 10 // 3 is 3, while -10 // 3 (-3.333) rounds down to -4.

Q8. [B] (Day 01: Division Operator) - The single slash division operator / always returns a float in Python 3, even if the division is exact.

Q9. [A] (Day 01: Exponentiation Operator) - The exponentiation operator ** has right-to-left associativity: 3 ** 2 = 9, then 2 ** 9 = 512.

Q10. [D] (Day 01: Modulo Operator) - The modulo operator % returns the remainder of division: 17 = 5 * 3 + 2, so 17 % 5 is 2.

Q11. [C] (Day 01: Identity vs Equality) - '==' checks value equality (__eq__), while 'is' checks if both variables refer to the exact same object in memory (id(a) == id(b)).

Q12. [B] (Day 01: Membership Operators) - For a list, the 'in' operator checks for exact item equality, not substrings. 'ban' is not an element, but 'banana' is.

Q13. [B] (Day 01: Type Casting) - int() cannot directly parse a string containing a decimal point ('45.67'). It must first be cast to float, or an integer string.

Q14. [D] (Day 01: Boolean Evaluation) - Any non-empty string in Python evaluates to True in a boolean context. The empty string '' evaluates to False.

Q15. [C] (Day 01: Ternary Operator) - Python uses the inline ternary syntax: 'x if condition else y'.

Q16. [A] (Day 01: Range Function) - range(start, stop, step) starts at 2, increments by 3, and stops before 9: [2, 5, 8].

Q17. [A] (Day 01: Range Defaults) - When a single argument is passed to range(stop), start defaults to 0 and step defaults to 1.

Q18. [D] (Day 01: Loop Control Statements) - break immediately terminates the innermost enclosing loop.

Q19. [D] (Day 01: Continue Statement) - For i in [0, 1, 2, 3, 4], even numbers (0, 2, 4) are skipped by continue. Odds added: 1 + 3 = 4.

Q20. [B] (Day 01: For-Else Construct) - A loop's else clause executes if and only if the loop exits naturally without hitting a break statement.

Q21. [A] (Day 01: Pass Statement) - pass is a null operation. It serves as a syntactic placeholder.

Q22. [D] (Day 01: Logical Operators) - (5 > 3) is True, (2 == 1) is False. True and False is False. not (False) is True.

Q23. [C] (Day 01: Bitwise Operators) - 5 is 0b101, 3 is 0b011. Bitwise AND: 101 & 011 = 001 (which is integer 1).

Q24. [B] (Day 01: Bitwise XOR) - ^ is bitwise XOR (not exponentiation). 6 is 110 in binary, 3 is 011. 110 ^ 011 = 101 (which is 5).

Q25. [A] (Day 01: Built-in type() and id()) - id(obj) returns the identity of an object, which is guaranteed to be unique and constant for its lifetime (in CPython, its memory address).

Q26. [D] (Day 02: String Immutability) - Python strings are strictly immutable. Modifying an individual character by index raises a TypeError.

Q27. [C] (Day 02: Basic Slicing) - Indices 1 up to (but excluding) 5: text[1]='R', text[2]='O', text[3]='G', text[4]='R'. Output: 'ROGR'.

Q28. [A] (Day 02: Negative Slicing) - Negative index -3 points to 'P'. text[-3:] extracts from index -3 to the end of the string: 'PER'.

Q29. [A] (Day 02: String Reversal) - s[::-1] starts from the end and steps backward by -1 across the entire string, producing the reversed string.

Q30. [D] (Day 02: String Stripping) - strip() removes leading and trailing whitespace characters (spaces, tabs, newlines) from both ends.

Q31. [C] (Day 02: String Splitting) - split() splits a string by a given delimiter and returns a list of strings.

Q32. [B] (Day 02: String Joining) - '-'.join(iterable) joins the elements of an iterable using '-' as the separator.

Q33. [B] (Day 02: find() vs index()) - str.find() safely returns -1 when a substring is not found, whereas str.index() raises a ValueError.

Q34. [D] (Day 02: String Case Methods) - title() capitalizes the first character of each word in the string.

Q35. [C] (Day 02: Raw Strings) - Prefixing a string with 'r' creates a raw string literal, where backslashes are treated as literal characters, not escape sequences.

Q36. [B] (Day 02: String Formatting (f-strings)) - {pi:.2f} formats the floating point number to 2 decimal places with standard rounding (3.14).

Q37. [A] (Day 02: String replace()) - Strings are immutable! replace() returns a new string; it does NOT modify msg in place. Since the return value wasn't reassigned, msg remains 'java is cool'.

Q38. [C] (Day 02: String startswith()) - str.startswith() accepts a tuple of prefixes and returns True if the string starts with any of them.

Q39. [C] (Day 02: String Character Check) - isdigit() returns True only if all characters in the string are digits. '1234a' contains 'a', so it returns False.

Q40. [B] (Day 02: String Repetition) - The multiplication operator * on a string repeats it the specified number of times: '======'.

Q41. [A] (Day 02: String count()) - 'banana' contains non-overlapping occurrences of 'an' at indices 1 and 3, so count('an') returns 2.

Q42. [D] (Day 02: String Multiline) - Triple quotes (''' or """) allow string literals to span multiple lines, preserving line breaks and indents.

Q43. [C] (Day 02: String Concatenation) - 'Hello ' + 'World' concatenates to 'Hello World', while 'Hello ' * 2 repeats the string to 'Hello Hello '.

Q44. [B] (Day 02: len() Function) - 'Hello' is 5 chars, '\n' is a single escape character (1 char), and 'World' is 5 chars. Total length = 5 + 1 + 5 = 11.

Q45. [A] (Day 02: String Formatting (% vs format)) - %.1f rounds 19.95 to 1 decimal place, producing 20.0.

Q46. [D] (Day 03: List Mutability) - Lists are mutable sequences. Item assignment by index modifies the element in place.

Q47. [D] (Day 03: Single Element Tuple) - In Python, parentheses alone do not create a tuple; a trailing comma is required: (42,). (42) is simply the integer 42.

Q48. [B] (Day 03: list.append() vs list.extend()) - append() adds the argument as a single element ([1, 2, [3, 4]], len=3). extend() unpacks and iterates over the elements ([1, 2, 3, 4], len=4).

Q49. [A] (Day 03: list.insert()) - insert(index, element) inserts the element before the given index, shifting subsequent elements to the right.

Q50. [D] (Day 03: list.pop()) - pop() without arguments removes and returns the last item in the list.

Q51. [D] (Day 03: list.remove()) - list.remove(x) searches for the first occurrence of x and deletes it. If x is not found, it raises a ValueError.

Q52. [B] (Day 03: Tuple Immutability) - Tuples are immutable sequences. Modifying elements by index raises a TypeError.

Q53. [A] (Day 03: Tuple Unpacking) - The elements of the tuple (10, 20, 30) are unpacked into variables a, b, c. Therefore, b = 20.

Q54. [A] (Day 03: Variable Swap) - Python evaluates the right side into a tuple (y, x) first and then unpacks it into the left side: x, y = y, x.

Q55. [B] (Day 03: List Comprehension Basics) - range(4) produces 0, 1, 2, 3. Multiplying each by 2 yields [0, 2, 4, 6].

Q56. [B] (Day 03: List Comprehension with Filter) - range(10) is 0 through 9. Numbers divisible by 3 (x % 3 == 0) are 0, 3, 6, and 9.

Q57. [A] (Day 03: list.sort() vs sorted()) - list.sort() is a method that sorts the list in place and returns None. sorted(iterable) is a built-in function that leaves the original iterable unchanged and returns a new list.

Q58. [C] (Day 03: list.index()) - index() returns the zero-based index of the FIRST occurrence of the specified value. The first 20 is at index 1.

Q59. [B] (Day 03: del Statement) - del nums[1:3] deletes elements from index 1 up to (excluding) 3, which are 2 and 3. Remaining elements: [1, 4].

Q60. [A] (Day 03: Built-in min, max, sum) - min is 1, max is 7, and sum is 4 + 1 + 7 + 2 = 14.

Q61. [C] (Day 04: Dictionary Key Requirements) - Dictionary keys must be hashable (immutable). Lists are mutable and unhashable, so they cannot serve as dictionary keys.

Q62. [A] (Day 04: Dictionary Access (.get)) - Direct indexing d[key] raises KeyError if key is missing. d.get(key, default) safely returns default (or None).

Q63. [B] (Day 04: dict.keys() & dict.values()) - d.keys() provides a view of keys ('a', 'b') and d.values() provides a view of values (1, 2).

Q64. [C] (Day 04: dict.pop()) - dict.pop(key, default) removes and returns the value for key; if key is not found and default is provided, it returns default.

Q65. [A] (Day 04: dict.update()) - update() overwrites matching keys from d2 into d1 and adds any new keys from d2.

Q66. [A] (Day 04: Empty Set vs Empty Dict) - Empty curly braces {} create an empty dictionary (dict). An empty set must be created using the constructor set().

Q67. [C] (Day 04: Set Uniqueness) - Sets only store unique elements. Duplicates are discarded, leaving {1, 2, 3, 4}, so len is 4.

Q68. [B] (Day 04: Set Operations (Union)) - The pipe operator | computes the union of two sets, returning all unique elements across both sets: {1, 2, 3, 4, 5}.

Q69. [A] (Day 04: Set Operations (Intersection)) - The ampersand operator & computes the intersection, returning elements present in both sets: {2, 3}.

Q70. [C] (Day 04: Set Operations (Difference)) - The minus operator - computes the set difference: elements in the first set but NOT in the second set: {1, 3}.

Q71. [C] (Day 04: set.discard() vs set.remove()) - s.remove(x) raises a KeyError if x is not in s. s.discard(x) removes x if present, and does nothing if absent.

Q72. [A] (Day 04: Dictionary Comprehension) - range(3) is 0, 1, 2. Each key x is mapped to x**2: {0: 0, 1: 1, 2: 4}.

Q73. [A] (Day 04: Iterating Dictionary) - Iterating directly over a dictionary defaults to iterating over its keys.

Q74. [A] (Day 04: dict.items()) - d.items() provides key-value pairs as (key, value) tuples.

Q75. [C] (Day 04: Frozenset) - frozenset is an immutable set. Because it is immutable, it is hashable and can be used as a dictionary key or an element of another set.

Q76. [C] (Day 05: Function Default Return) - In Python, any function that does not explicitly return a value returns None by default.

Q77. [A] (Day 05: *args and **kwargs) - *args collects extra positional arguments into a tuple, while **kwargs collects extra keyword arguments into a dict.

Q78. [A] (Day 05: Default Parameters) - SyntaxError: non-default parameter follows default parameter. Parameters with default values must appear AFTER all parameters without defaults.

Q79. [C] (Day 05: Lambda Syntax) - Lambda functions use the syntax: lambda arguments: expression.

Q80. [B] (Day 05: Built-in map()) - map() applies the function (adding 1) to each item in the iterable, resulting in [2, 3, 4].

Q81. [A] (Day 05: Built-in filter()) - filter() retains only those elements for which the predicate returns True (x > 0), giving [3, 5].

Q82. [D] (Day 05: Built-in zip()) - zip() aggregates corresponding elements from each iterable into pairs of tuples: [('a', 1), ('b', 2)].

Q83. [B] (Day 05: Built-in enumerate()) - enumerate() produces 2-tuples containing a 0-based counter index and the corresponding value: [(0, 'apple'), (1, 'banana')].

Q84. [B] (Day 05: global Keyword) - Without the global declaration, assigning to x inside a function creates a new local variable. global tells Python to modify the module-level variable.

Q85. [D] (Day 05: Regular Expressions (re.search vs re.match)) - re.match() checks for a match only at the start of the string, while re.search() scans through the entire string for a match.

Q86. [D] (Day 05: RegEx Metacharacters) - \d matches any Unicode decimal digit (equivalent to [0-9] in ASCII).

Q87. [C] (Day 05: RegEx findall()) - re.findall() returns all non-overlapping matches of pattern in string as a list of strings: ['10', '25'].

Q88. [A] (Day 05: Built-in all() and any()) - any() returns True if ANY element is truthy (all are falsy here, so False). all() returns True if ALL elements are truthy (all are truthy here, so True).

Q89. [A] (Day 06: OOP self Parameter) - By convention, self represents the instance of the class on which the method is called, allowing access to instance attributes.

Q90. [A] (Day 06: OOP Initializer __init__) - __init__ is the initializer method called immediately after memory is allocated (__new__) to initialize instance attributes.

Q91. [C] (Day 06: Instance vs Class Variables) - school is a class variable defined outside __init__. It is shared across all instances of Student.

Q92. [B] (Day 06: Basic Inheritance) - Python specifies base classes inside parentheses after the class name: class Dog(Animal):.

Q93. [D] (Day 06: Built-in isinstance()) - isinstance(obj, class_or_tuple) returns True if obj is an instance of any class in the tuple. Since 5 is an int, it returns True.

Q94. [D] (Day 06: String Representation __str__) - print(obj) and str(obj) call obj.__str__() to obtain a human-readable string representation of the object.

Q95. [C] (Day 07: File open() Default Mode) - The default mode for open() is 'r' (read text mode).

Q96. [B] (Day 07: Context Manager 'with') - The 'with' statement acts as a context manager (__enter__ and __exit__), guaranteeing that file descriptors are safely closed upon block exit.

Q97. [A] (Day 07: JSON Serialization) - json.dumps() (dump string) serializes a Python object to a JSON string. json.dump() writes directly to a file stream.

Q98. [D] (Day 09: Flask Application Setup) - In Flask, @app.route('/path') is the standard decorator used to map a URL pattern to a view handler function.

Q99. [C] (Day 10: HTTP Status Codes) - HTTP 201 Created indicates that the request was successful and led to the creation of a new resource on the server.

Q100. [C] (Day 11: Pandas DataFrame Inspection) - df.head() returns the first n rows (default n=5) of the DataFrame.

Q101. [A] (Day 01: Short-Circuit Evaluation) - 'or' returns the first truthy operand ('Python') or the last operand. 'and' returns the first falsy operand (0) or the last operand. [1, 2] is truthy, so c is [1, 2].

Q102. [C] (Day 02: Negative Stride Slicing) - text[4] is 'O', text[3] is 'H', text[2] is 'T'. Stop index 1 ('Y') is exclusive. With step -1, it outputs 'OHT'.

Q103. [D] (Day 02: String Immutability in Pipelines) - Because strings are immutable, s.upper() and s.replace() return new strings without modifying s. s remains 'python programming'. Calling s.title() results in 'Python Programming'.

Q104. [B] (Day 01: Loop Else with Break) - When n = 4, the condition is met and break is executed. When a loop is exited via break, its else clause is skipped.

Q105. [D] (Day 01: Chained Comparison Operators) - Python evaluates chained comparisons as: (1 < 3) and (3 > 2) and (2 == 2). All three are True, so the result is True.

Q106. [C] (Day 01: Bitwise Shift Operations) - 8 >> 2 shifts right by 2 bits: 8 // 4 = 2. 3 << 2 shifts left by 2 bits: 3 * 4 = 12. 2 + 12 = 14.

Q107. [D] (Day 02: String find() with Start Index) - find(sub, start) begins searching at index 4 ('c'). The next 'a' occurs at index 5 ('abra[c]a...').

Q108. [A] (Day 02: String split() with maxsplit) - maxsplit=2 performs at most 2 splits, yielding a list with 3 elements: ['a', 'b', 'c:d'].

Q109. [D] (Day 02: String Stripping with Specific Characters) - strip('wn.') removes any characters from the set {'w', 'n', '.'} from both ends. 'www.' is stripped from left, '.in' is stripped from right, leaving 'cdac'.

Q110. [C] (Day 01: Integer Caching / Small Integer Interning) - CPython pre-allocates integers from -5 to 256, so x is y is True. a == b compares values (257 == 257), which is always True.

Q111. [C] (Day 02: String zfill and rjust) - zfill(width) pads the string on the left with ASCII '0' digits until the given width is reached: '00042'.

Q112. [B] (Day 01: Operator Precedence) - 4 * 2 = 8; 3 + 8 = 11; 11 > 10 is True. 2 == 3 is False; not False is True. True and True is True.

Q113. [A] (Day 02: f-string Expression Specifier) - Python 3.8+ introduced the '=' specifier in f-strings for self-documenting expressions, producing 'x=10'.

Q114. [D] (Day 01: While Loop Decrement) - Initially i=3; first iteration decrements to 2 (prints 2); second iteration decrements to 1 (prints 1); third decrements to 0 (prints 0). Then i > 0 fails. Output: 2 1 0.

Q115. [C] (Day 02: String partition()) - str.partition(sep) splits at the first occurrence of sep and returns a 3-tuple: (before, sep, after).

Q116. [A] (Day 03: Shallow Copy Mechanics) - a.copy() creates a shallow copy. The outer list b has len=3 after append, but a still has len=2. However, inner lists are shared references, so mutating b[0][1] mutates a[0][1] to 99.

Q117. [A] (Day 03: Nested List Comprehension) - The order of for-clauses matches standard nested loops: for row in matrix: for x in row: if x % 2 == 0. Even elements extracted are [2, 4, 6, 8].

Q118. [D] (Day 03: List Multiplication Pitfall) - Multiplying [[0] * 2] * 2 duplicates the reference to the same inner list. Mutating grid[0][0] reflects in both rows: [[5, 0], [5, 0]].

Q119. [C] (Day 04: Dictionary Dynamic Views) - In Python 3, dict.keys() returns a dynamic view object that reflects changes to the underlying dictionary immediately. Converting to list after mutations gives ['notebook', 'pencil'].

Q120. [B] (Day 04: dict.setdefault()) - If the key already exists, setdefault() returns its existing value (1 for 'a'). If missing, it inserts the key with the default value and returns it (200 for 'b').

Q121. [A] (Day 04: Dictionary Merge / Unpacking) - In {**d1, **d2}, keys from later dictionaries overwrite keys from earlier ones. 'y' becomes 99.

Q122. [D] (Day 04: Dictionary fromkeys() Pitfall) - dict.fromkeys(keys, value) associates each key with the exact same default object reference. Appending to d['a'] modifies the shared list for all keys.

Q123. [C] (Day 03: Modifying List During Iteration) - Modifying a list during iteration shifts the indices of remaining elements. When 1 is removed, 2 moves to index 0 and is skipped on the next step. 3 is removed, 4 is skipped, and 5 is never checked. Result: [2, 4, 5].

Q124. [B] (Day 03: Tuple with Mutable List) - The tuple's structure (references) remains immutable, but the object at t[1] is a mutable list. Calling .append() mutates the list in place without modifying the tuple reference itself.

Q125. [A] (Day 04: Set Symmetric Difference) - The caret operator ^ computes symmetric difference: elements in either set, but not in both. {1, 2, 3} ^ {2, 3, 4} = {1, 4}.

Q126. [C] (Day 03: List Extended Slicing Assignment) - Slice assignment replaces the slice [1:4] (elements 2, 3, 4) with the elements from [20, 30]. The list becomes [1, 20, 30, 5].

Q127. [C] (Day 03: list.sort() with Key Function) - len('pie')=3, len('apple')=5, len('banana')=6. Sorted by length in ascending order: ['pie', 'apple', 'banana'].

Q128. [B] (Day 04: Dictionary Popitem) - In Python 3.7+, dictionaries preserve insertion order. popitem() removes and returns the last (LIFO) key-value pair as a (key, value) tuple: ('c', 3).

Q129. [A] (Day 03: list.count() vs in) - In Python, bool is a subclass of int: True == 1 and False == 0. nums.count(1) counts True and 1 (total 2); nums.count(0) counts 0 and False (total 2).

Q130. [C] (Day 04: Set issubset & issuperset) - {1, 2} is a subset of {1, 2, 3}, and {1, 2, 3} is a superset of {1, 2}. Both expressions evaluate to True.

Q131. [C] (Day 05: Mutable Default Argument Pitfall) - Default argument expressions are evaluated once at function definition time. list1 and list3 share the same default list object. list2 was passed an explicit new list []. Output: True ['CPU', 'RAM'] ['GPU'].

Q132. [B] (Day 05: Higher-Order Functions (map + filter)) - filter keeps odd numbers: [1, 3, 5]. map multiplies each by 10: [10, 30, 50].

Q133. [A] (Day 05: Regular Expressions match vs search) - re.match() matches only at the beginning of the string ('Order' does not match \d+), returning None (m1 is None is True). re.search() scans the entire string and finds '98452'.

Q134. [A] (Day 05: Keyword-Only Arguments (*)) - The bare asterisk * in the parameter list indicates that all subsequent parameters are keyword-only arguments.

Q135. [C] (Day 05: Positional-Only Arguments (/)) - In Python 3.8+, parameters before '/' are positional-only. Calling add(a=1, b=2, c=3) raises a TypeError.

Q136. [B] (Day 05: nonlocal Keyword) - The nonlocal keyword binds x to the nearest enclosing scope (in outer), allowing inner() to modify outer's x to 15.

Q137. [A] (Day 05: functools.reduce()) - reduce applies the function cumulatively: ((1 * 2) * 3) * 4 = 24.

Q138. [D] (Day 05: RegEx Grouping and sub()) - \1 matches '10', \2 matches '25'. The replacement r'\2/\1' swaps them to '25/10'.

Q139. [B] (Day 05: Lambda Sorting by Multiple Keys) - Grade 'A' comes before 'B'. For grade 'A', -s[1] sorts age descending: Alice (-22) comes before Charlie (-19). Output: ['Alice', 'Charlie', 'Bob'].

Q140. [B] (Day 05: Unpacking in Function Calls) - display(*args, **kwargs) unpacks a=5, b=15, c=20. 5 + 15 + 20 = 40.

Q141. [A] (Day 05: Recursion Base Case) - Python has a call stack depth limit (typically 1000). Exceeding it raises a RecursionError.

Q142. [C] (Day 05: RegEx re.split()) - The pattern [,;\s]+ matches one or more commas, semicolons, or whitespace characters, neatly splitting all words.

Q143. [C] (Day 05: Function Annotation Attributes) - Function annotations are stored in the function's __annotations__ dictionary attribute.

Q144. [A] (Day 05: Basic Decorator Syntax) - The @my_decorator syntax is syntactic sugar for reassigning func = my_decorator(func).

Q145. [A] (Day 05: Scope LEGB Rule) - Python resolves variables using LEGB: Local scope first, then Enclosing functions, then Global (module level), then Built-in.

Q146. [D] (Day 06: Class Attribute Shadowing) - Device.count is incremented twice (count=2). Assigning d1.count = 10 creates an instance attribute on d1 that shadows the class attribute. d2.count and Device.count still access the class attribute (2).

Q147. [B] (Day 06: Property Setter with Validation) - w.balance -= 100 evaluates to w.balance = 50 - 100 = -50, which raises ValueError in setter before modifying __balance. In the except block, w.balance += 20 modifies 50 to 70. w.balance prints 70.

Q148. [B] (Day 06: Name Mangling for Private Attributes) - Python enforces private variables with leading double underscores via name mangling: _ClassName__var, making it _Account__secret.

Q149. [D] (Day 06: @classmethod vs @staticmethod) - @classmethod receives the class object (cls) as its first implicit parameter. @staticmethod is a plain function scoped inside the class namespace, receiving neither self nor cls.

Q150. [D] (Day 06: Dunder __repr__ vs __str__) - repr(obj) calls __repr__ (unambiguous developer representation). str(obj) calls __str__ (human-readable string); if __str__ is not implemented, Python falls back to __repr__.

Q151. [C] (Day 06: Dunder __len__ Protocol) - Python requires __len__() to return a non-negative integer (>= 0). Returning a negative number or non-integer raises a TypeError or ValueError.

Q152. [C] (Day 06: Operator Overloading (__add__)) - v1 + v2 calls v1.__add__(v2), which adds coordinates: 1+3=4 and 2+4=6. Output: 4 6.

Q153. [A] (Day 06: super() with Single Inheritance) - super().__init__() calls parent class A's constructor, setting self.val = 10. Then self.val += 5 increases it to 15.

Q154. [D] (Day 06: Dunder __eq__ and Hashability) - In Python 3, defining a custom __eq__ without defining __hash__ sets __hash__ to None, making instances unhashable and raising TypeError if added to a set or dict key.

Q155. [B] (Day 06: Polymorphism and Method Overriding) - Python uses duck typing: as long as each object implements .speak(), the function calls their respective method implementations.

Q156. [C] (Day 07: File seek() and tell()) - seek(4) places the cursor at index 4 (0:'a', 1:'b', 2:'c', 3:'d', 4:'e'). read(3) reads 3 chars: 'efg'. The file pointer is now at index 7. tell() returns 7.

Q157. [B] (Day 07: Pickle File Mode Requirement) - pickle serializes Python objects into a binary byte stream. Thus, the file must be opened in write-binary mode ('wb').

Q158. [A] (Day 07: csv.DictReader Structure) - csv.DictReader maps the fieldnames in the first row to each subsequent row's values, returning each row as an OrderedDict/dict.

Q159. [B] (Day 07: json.loads vs json.load) - json.loads(s) (load string) deserializes a JSON string into a Python object. json.load(fp) deserializes directly from an open file/stream.

Q160. [B] (Day 07: SQLite Parameterized Queries) - Parameterized queries separate SQL code from untrusted user data, preventing SQL Injection vulnerabilities.

Q161. [A] (Day 07: SQLite commit() Requirement) - By default, sqlite3 starts transactions implicitly. Any uncommitted data modifications (INSERT/UPDATE/DELETE) are rolled back if the connection is closed without calling conn.commit().

Q162. [D] (Day 07: File 'r+' Mode) - 'r+' opens an existing file for both reading and writing without truncating it. Writing begins at the current file pointer position.

Q163. [C] (Day 07: File 'w' Truncation) - Mode 'w' truncates the file immediately upon opening, erasing any existing contents.

Q164. [A] (Day 07: File readline() Trailing Newline) - readline() includes the trailing newline character '\n' (unless it's the last line without one). It returns an empty string '' when EOF is reached.

Q165. [A] (Day 07: SQLite fetchall() Return Type) - By default, cursor.fetchall() returns a list of tuples containing column values.

Q166. [D] (Day 08: Generator Function Invocation) - Calling a generator function returns a generator object without executing any code inside the function until next() or a loop is invoked.

Q167. [B] (Day 08: Generator next() and StopIteration) - The generator yields 1, then 2. The third call to next(c) exhausts the generator, raising StopIteration, caught to print 'Done'.

Q168. [B] (Day 08: Custom Iterator Protocol) - The iterator protocol requires __iter__() (returning self) and __next__() (returning the next item or raising StopIteration).

Q169. [A] (Day 08: Generator Expression Memory Efficiency) - Generator expressions produce items on-demand (lazy evaluation), maintaining a minimal memory footprint.

Q170. [D] (Day 08: itertools.chain()) - itertools.chain() links multiple iterables consecutively into a single sequence: [1, 2, 'a', 'b'].

Q171. [C] (Day 08: yield from Syntax) - yield from delegates iteration to a sub-generator or iterable, yielding each element individually (10, then 20, then 30).

Q172. [B] (Day 08: Built-in iter() with Sentinel) - iter(callable, sentinel) creates an iterator that invokes callable on each next() call until the returned value equals sentinel.

Q173. [A] (Day 08: Generator State Persistence) - Generators freeze and maintain their local stack frame between yield statements. count increments on each resumption: 1, 2, 3.

Q174. [D] (Day 08: Exception Handling (try-except-else-finally)) - A return statement in a finally block always overrides any return statement executed in the try or except blocks. Both calls return 'CLEANUP_DONE'.

Q175. [C] (Day 08: Try-Except-Else Block) - The else block of a try statement executes only if the try clause finishes without raising any exception.

Q176. [A] (Day 09: Flask Route Dynamic Parameters) - Flask uses angle brackets with type converters: @app.route('/books/<int:book_id>').

Q177. [A] (Day 09: Flask HTTP Methods Configuration) - By default, Flask routes listen only to GET requests (and implicitly HEAD/OPTIONS). To accept POST, methods=['GET', 'POST'] must be declared.

Q178. [D] (Day 09: MVC vs MVT Pattern) - In Django MVT, the View (views.py) contains business logic and controls data flow, corresponding to the Controller in MVC. The Template corresponds to the View.

Q179. [B] (Day 10: BeautifulSoup Scraping Tag & Attribute) - tag.text extracts the visible inner text ('Click Here'), and tag['href'] accesses the HTML attribute value ('https://example.com').

Q180. [B] (Day 10: REST API Status Code 204) - 204 No Content signifies that the action was successfully fulfilled, but there is no payload body to return (standard for successful DELETE).

Q181. [C] (Day 10: HTTP Verbs Idempotency) - An idempotent method produces the same result on multiple identical requests. GET, PUT, and DELETE are idempotent. POST creates new resources on each call and is non-idempotent.

Q182. [D] (Day 10: Flask request.get_json()) - request.get_json() parses the incoming JSON request payload into a native Python dictionary or list.

Q183. [C] (Day 10: BeautifulSoup find_all()) - find_all() returns a list of all matching Tag objects matching the tag name and attributes.

Q184. [A] (Day 10: Flask jsonify() Helper) - jsonify() serializes the Python object into JSON format and sets the HTTP response mimetype header to 'application/json'.

Q185. [A] (Day 10: HTTP 405 Status Code) - 405 Method Not Allowed indicates the resource exists, but does not support the HTTP verb used by the client.

Q186. [C] (Day 10: NumPy 2D Array Slicing) - arr[:2, 1:] slices rows 0 and 1, and columns from index 1 onward (columns 1 and 2), giving [[20, 30], [50, 60]].

Q187. [B] (Day 10: NumPy reshape() with -1) - Passing -1 to reshape tells NumPy to infer that dimension automatically: 12 / 3 = 4 columns. Shape is (3, 4).

Q188. [B] (Day 11: Pandas Boolean Filtering Syntax) - Python's logical 'and' expects a single scalar boolean, raising 'The truth value of a Series is ambiguous'. Element-wise boolean combination requires bitwise '&' and parentheses around each condition.

Q189. [A] (Day 11: Pandas .loc vs .iloc) - .loc selects by index/column labels; .iloc selects strictly by integer position.

Q190. [C] (Day 11: Pandas groupby() Aggregation) - Two unique companies ('Duplo', 'Lego') yield 2 rows. Three aggregated columns: Price mean, Price max, Units sum -> shape (2, 3). Max price for Lego is 30.0.

Q191. [C] (Day 10: NumPy Boolean Masking) - arr % 2 != 0 creates a boolean mask [True, False, True, False, True]. Indexing arr with this mask extracts [1, 3, 5].

Q192. [A] (Day 11: Pandas Handling Missing Values) - dropna() drops rows (or columns) containing null values; fillna(val) substitutes missing values with val.

Q193. [A] (Day 11: Pandas Series Creation) - Pandas assigns a default integer RangeIndex starting at 0: 0, 1, 2.

Q194. [D] (Day 11: Matplotlib subplots() Return Values) - plt.subplots() returns a Figure object (the top-level container) and an Axes object (or array of Axes) where plotting occurs.

Q195. [C] (Day 11: Matplotlib plt.barh()) - barh() draws horizontal bars (categories along the y-axis, values along the x-axis).

Q196. [B] (Day 10: NumPy Broadcasting Rules) - NumPy broadcasts array b across each row of a. The resulting array retains shape (2, 3).

Q197. [A] (Day 11: Pandas value_counts()) - value_counts() computes frequencies of each unique category in descending order.

Q198. [D] (Day 10: Web Scraping robots.txt) - robots.txt outlines scraping guidelines and policies for automated web crawlers.

Q199. [D] (Day 10: Requests Library Response JSON) - resp.json() is the built-in convenience method that decodes resp.text with json.loads(). Both are valid.

Q200. [B] (Day 11: Matplotlib plt.savefig() Placement) - plt.show() resets the current figure state upon window close. Calling savefig() after show() often saves a blank canvas.

Q201. [A] (Day 01: In-Place Modification inside Tuple) - In Python, 't[1] += [4]' executes list.extend([4]) in-place (which succeeds, modifying the list to [2, 3, 4]), and then attempts to assign the result back to t[1] (which raises TypeError because tuples are immutable). Thus, the error is caught, and t shows (1, [2, 3, 4]).

Q202. [C] (Day 01: Walrus Operator (:=)) - The walrus operator := assigns y = x * 2 within the comprehension condition. When y > 6 (for x=4: 8, x=5: 10, x=6: 12), y is yielded into res, producing [8, 10, 12].

Q203. [B] (Day 01: Structural Pattern Matching) - Python's match/case unpacks the 3-element tuple into the pattern ('move', x, y), binding x=10 and y=20, producing 'Moving to 10,20'.

Q204. [B] (Day 01: IEEE-754 Floating Point Precision) - Due to binary IEEE-754 floating point representation, 0.1 + 0.2 equals 0.30000000000000004, making 0.1 + 0.2 == 0.3 False. Rounding to 2 decimal places yields 0.3, so the rounded comparison is True.

Q205. [D] (Day 01: Starred Expression Unpacking) - Extended unpacking binds the first element to first (10), the last element to last (50), and collects all intermediate elements into a list: middle = [20, 30, 40].

Q206. [C] (Day 02: Advanced String Formatting Specifier) - The format specifier ':*>15,._2f' specifies: align right (>), pad with asterisk (*), total width 15, comma thousands separator (,), and 2 decimal places (.2f). '1,234,567.89' has length 12; padded to 15 chars gives '***1,234,567.89'.

Q207. [C] (Day 02: String Interning Mechanics) - CPython automatically interns compile-time constant string literals that look like valid identifiers (alphanumeric + underscore). Dynamically constructed strings or those with special characters are not interned unless sys.intern() is explicitly called.

Q208. [B] (Day 02: str.casefold() vs str.lower()) - casefold() is designed for aggressive Unicode caseless matching and handles characters like the German lowercase 'ß' (converting it to 'ss'), which lower() does not change.

Q209. [A] (Day 01: Short-Circuit Side Effects) - The first call to check() appends 1 and returns False. Because the 'and' operator short-circuits on the first falsy operand, the second check() is never invoked. len(nums) is 1.

Q210. [D] (Day 02: bytes vs bytearray) - bytes objects are immutable sequences of single bytes (integers in 0-255). bytearray objects are mutable sequences of bytes supporting in-place modifications.

Q211. [B] (Day 03: copy.deepcopy() Mechanics) - copy.deepcopy maintains an internal memo dictionary mapping original objects to their copies to handle circular references. b[2] points to b itself (True) and not the original a (False).

Q212. [B] (Day 04: Hashable Contract for Dictionary Keys) - Dictionary lookup checks hash(key) first to find the bucket, and then checks equality (key == target) to handle collisions. Identical keys require hash(a) == hash(b) and a == b.

Q213. [A] (Day 04: collections.defaultdict) - When a missing key is accessed, defaultdict calls the default_factory callable (list()) to automatically create and insert an empty list [].

Q214. [D] (Day 04: collections.Counter Addition) - Adding Counters sums corresponding item counts: a: 3+1=4, b: 1+2=3, c: 0+4=4. Output: 4 3 4.

Q215. [C] (Day 03: List Comprehension Scope in Python 3) - In Python 3, list comprehensions have their own local function-like scope. The loop variable x inside the comprehension does NOT leak into the outer scope. Outer x remains 100.

Q216. [B] (Day 04: Custom Class as Dict Key) - Because Point implements both __hash__ and __eq__ based on (x, y), Point(1, 2) hashes to the same value and compares equal to the key in d, returning 'Origin'.

Q217. [A] (Day 04: Set Hash Collisions) - In Python, 1 == True == 1.0, and hash(1) == hash(True) == hash(1.0). Since sets deduplicate based on hash equality and value equality, only one element remains. len is 1.

Q218. [D] (Day 03: collections.namedtuple) - namedtuple creates a tuple subclass that allows attribute access by field name (t.title) as well as index access (t[1]), while retaining the immutability and compact memory footprint of tuples.

Q219. [C] (Day 03: collections.deque Performance) - A Python list is a contiguous array where inserting or popping at index 0 requires shifting all remaining n elements (O(n)). deque is a doubly-linked list of blocks with O(1) appends and pops at both ends.

Q220. [A] (Day 04: Dictionary Keys Union in Python 3) - dict.keys() returns a dictionary view that behaves like a set (set-like view), supporting set operations like & (intersection). The common key is {'b'}.

Q221. [A] (Day 05: Closure Late Binding Trap) - Python closures bind variables by reference (late binding), not by value. When the lambdas are executed, the loop has completed and i is 3 for all of them. f(2) = 2 * 3 = 6 for each function: [6, 6, 6, 6].

Q222. [D] (Day 05: Fixing Closure Late Binding) - Using a default parameter 'i=i' forces Python to evaluate and bind the current value of i at lambda definition time into a local parameter.

Q223. [C] (Day 05: functools.wraps Role) - Without @functools.wraps, decorated functions lose their identity: __name__ becomes 'wrapper' and __doc__ is erased. wraps restores the original function's introspection attributes.

Q224. [C] (Day 05: Decorator with Arguments (Decorator Factory)) - A decorator taking arguments requires 3 levels: (1) outer factory accepting decorator parameters, (2) middle decorator accepting the target function, and (3) inner wrapper executing logic and arguments.

Q225. [A] (Day 05: Multiple Decorator Stacking Order) - Decorators are applied bottom-up: dec2 wraps func, then dec1 wraps the result: dec1(dec2(func)). When calling func(), execution flows from outermost (dec1) to innermost (dec2).

Q226. [D] (Day 05: functools.lru_cache) - lru_cache implements Least Recently Used caching (memoization), storing computed outputs mapped to argument hashes and avoiding redundant recursive calculations.

Q227. [C] (Day 05: RegEx Lookaround Assertions) - (?=\s*USD) is a positive lookahead. It asserts that \d+ is followed by USD without consuming USD into the matched string. Result: ['450'].

Q228. [B] (Day 05: RegEx Negative Lookbehind) - (?<!\$) is a negative lookbehind asserting that the number is NOT preceded by a dollar sign. 100 is preceded by $, so it is ignored. Output: ['200'].

Q229. [D] (Day 05: RegEx Named Capturing Groups) - Named capturing groups (?P<name>...) are accessed by passing the group name string to match.group('name') or via match.groupdict().

Q230. [D] (Day 05: Modifying Global vs UnboundLocalError) - Because count += 1 assigns to count, Python marks count as a local variable throughout the entire function body. Referencing it before the assignment triggers UnboundLocalError.

Q231. [C] (Day 06: MRO & Diamond Inheritance) - According to Python's C3 Linearization algorithm, D's MRO is [D, B, C, A, object]. Looking up ping() finds B's implementation first, returning 'B'.

Q232. [B] (Day 06: Cooperative super() in Diamond Hierarchy) - MRO for D is [D, B, C, A, object]. In B, super().process() resolves to the next class in D's MRO, which is C! C calls super() to A. Output: ['D', 'B', 'C', 'A'].

Q233. [A] (Day 06: __new__ vs __init__ (Singleton Pattern)) - __new__(cls) allocates the raw memory and returns the newly created object. __init__(self) is then invoked with that instance to set initial attribute values.

Q234. [C] (Day 06: Memory Optimization with __slots__) - __slots__ replaces the dynamic per-instance __dict__ with fixed-size descriptor arrays, significantly reducing memory consumption and preventing arbitrary attribute assignment.

Q235. [C] (Day 06: Abstract Base Classes (abc.ABC)) - Python strictly prevents instantiation of any subclass that does not override all abstract methods, raising a TypeError during instantiation.

Q236. [A] (Day 06: Dunder __call__ Method) - Implementing __call__ makes an instance callable like a standard function. m(10) calls m.__call__(10) = 5 * 10 = 50.

Q237. [A] (Day 06: Descriptor Protocol) - The Descriptor Protocol consists of __get__(self, instance, owner), __set__(self, instance, value), and __delete__(self, instance). It powers properties, methods, and classmethods.

Q238. [D] (Day 06: Metaclass Definition) - A metaclass is a class whose instances are classes. In Python, the default metaclass for all classes is 'type'.

Q239. [C] (Day 06: Context Manager Protocol (__enter__ and __exit__)) - If __exit__ returns True (or a truthy value), Python swallows the exception and continues execution after the with block without propagating it.

Q240. [B] (Day 06: Dunder __getitem__ and Iteration) - If __iter__ is not implemented, Python attempts to iterate by calling __getitem__ with integer indices 0, 1, 2... until an IndexError is raised, yielding [0, 10, 20].

Q241. [A] (Day 07: File seek() with os.SEEK_END) - os.SEEK_END (value 2) references the end of the file. A negative offset moves the pointer backward from the EOF.

Q242. [D] (Day 07: Custom JSONEncoder) - Standard json does not serialize datetime objects natively. Passing default=str or subclassing json.JSONEncoder converts unsupported objects.

Q243. [C] (Day 07: Pickle Security Vulnerability) - pickle is not secure against erroneous or maliciously constructed data. The __reduce__ protocol allows pickled data to construct and execute arbitrary OS commands (e.g. os.system) upon unpickling.

Q244. [A] (Day 07: SQLite sqlite3.Row Factory) - sqlite3.Row provides a highly optimized dictionary-like interface, allowing column access both by case-insensitive column name and by integer index.

Q245. [A] (Day 07: SQLite Connection as Context Manager) - Using 'with conn:' manages the transaction (commits on success, rolls back on exception). It does NOT close the connection; conn.close() must still be called separately.

Q246. [D] (Day 07: In-Memory SQLite Database) - Passing ':memory:' as the database path opens a private, pure-RAM database that exists only as long as the connection is open.

Q247. [C] (Day 07: contextlib.@contextmanager) - The value yielded by the generator is passed to the 'as' variable in the 'with' statement. Any cleanup code in finally runs when the with block terminates.

Q248. [A] (Day 07: io.StringIO vs io.BytesIO) - io.StringIO provides an in-memory file-like buffer for strings (unicode text), while io.BytesIO provides an in-memory buffer for raw bytes.

Q249. [D] (Day 07: csv.writer lineterminator Parameter) - By default, csv.writer terminates rows with '\r\n'. If open() does not specify newline='', Python's universal newline translation replaces '\n' with '\r\n', creating '\r\r\n' (blank lines between rows).

Q250. [D] (Day 07: SQLite Executemany Performance) - executemany() prepares the SQL statement once and executes it across all tuples in batch, significantly improving performance.

Q251. [C] (Day 08: Bidirectional Generator .send()) - next(gen) advances to the first yield total (0). gen.send(10) sets val=10, total becomes 10, yields 10. gen.send(20) sets val=20, total becomes 30, yields 30. Output: 10 30.

Q252. [A] (Day 08: Priming a Generator Coroutine) - A newly created generator has not yet reached its first yield expression. Calling send(val) with a non-None value before the generator has yielded raises TypeError.

Q253. [A] (Day 08: Generator .close() Method) - Calling gen.close() raises a GeneratorExit exception at the point where the generator was suspended, allowing try-finally cleanup blocks to execute.

Q254. [D] (Day 08: yield from Return Value) - yield from yields each item from sub_gen (1, 2). When sub_gen returns 'COMPLETED' (carried via StopIteration.value), that return value is assigned to res. main_gen then yields res, resulting in [1, 2, 'COMPLETED'].

Q255. [B] (Day 08: itertools.islice with Generators) - itertools.islice lazily extracts the first 3 items from the generator without trying to evaluate the infinite sequence to completion: [10, 12, 14].

Q256. [B] (Day 08: inspect.isgeneratorfunction) - During compilation to bytecode, if the function body contains the yield keyword, the code object's flags are marked with CO_GENERATOR.

Q257. [D] (Day 08: Custom Iterable Class) - An iterable is any object capable of returning its members one at a time (implements __iter__). An iterator is the stateful stream object that actually produces values on __next__().

Q258. [D] (Day 08: StopIteration Value Attribute) - In Python 3.3+, a value returned from a generator is attached to the .value attribute of the StopIteration exception that terminates the generator.

Q259. [C] (Day 08: Generator Expression Re-use) - Generators are single-use iterators. Once exhausted, subsequent iterations immediately return an empty result without resetting: [1, 2] [].

Q260. [A] (Day 08: Exception Handling Hierarchy) - In Python's exception hierarchy, BaseException is the root. SystemExit, KeyboardInterrupt, and GeneratorExit inherit directly from BaseException so standard 'except Exception' handlers do not accidentally catch them.

Q261. [D] (Day 09: Flask Application vs Request Context) - Flask splits context into two stacks: Application Context (manages app-level objects like db configs and 'g') and Request Context (manages request-specific info like URLs, headers, and form data).

Q262. [D] (Day 09: Flask Blueprints Purpose) - Blueprints enable modular architecture by allowing route handlers, templates, and static folders to be grouped and registered under specific URL prefixes.

Q263. [C] (Day 10: REST PUT vs PATCH Semantics) - PUT performs complete resource replacement (idempotent). PATCH applies a partial update/delta to modify only specific fields.

Q264. [A] (Day 10: HTTP 401 vs 403 Status Codes) - 401 Unauthorized indicates authentication is missing or invalid. 403 Forbidden indicates the identity is known, but access is refused due to insufficient permissions.

Q265. [A] (Day 10: Flask Custom Error Handler) - @app.errorhandler(404) registers a function to intercept HTTP 404 exceptions and return a custom response and status code.

Q266. [D] (Day 10: BeautifulSoup CSS Selector select()) - soup.select() evaluates CSS selectors and returns a list of all matching Tag elements.

Q267. [C] (Day 10: Web Scraping User-Agent Header) - Web servers frequently reject requests bearing the default 'python-requests' User-Agent. Spoofing a standard browser User-Agent avoids bot blocking.

Q268. [B] (Day 10: requests.Session() Connection Pooling) - requests.Session() uses urllib3 connection pooling, persisting cookies across requests and keeping TCP/TLS connections alive, dramatically speeding up consecutive requests.

Q269. [D] (Day 10: BeautifulSoup NavigableString vs Tag) - find('p') returns a bs4.element.Tag object representing the HTML node. .string extracts the inner content as a bs4.element.NavigableString.

Q270. [B] (Day 10: REST Architectural Constraints) - The 6 REST constraints are: Client-Server, Stateless, Cacheable, Uniform Interface, Layered System, and Code on Demand. ORM is an internal implementation pattern, not a REST constraint.

Q271. [B] (Day 10: NumPy 3D Array Striding & Slicing) - Axis 0 has length 2. Axis 1 is reversed (length 3). Axis 2 slices indices 1 and 2 (length 2). Shape is (2, 3, 2). sub[0, 0, 0] corresponds to original arr[0, 2, 1] (row index 2 from reverse, column 1): 2*4 + 1 = 9.

Q272. [B] (Day 10: NumPy View vs Copy Slicing) - Basic slicing in NumPy creates a memory view (sharing the underlying data buffer), not a copy. Modifying elements in slice b modifies original array a (a[1] becomes 99).

Q273. [D] (Day 10: NumPy Fancy Indexing creates Copy) - Fancy indexing (indexing via lists/arrays of integer indices or boolean masks) always allocates and returns a new array copy, unlike basic slicing which returns a view.

Q274. [D] (Day 11: Pandas SettingWithCopyWarning) - Chained assignment performs two separate indexing operations. If the intermediate step is a copy, the assignment modifies a temporary object and is lost. Use df.loc[df['status'] == 'active', 'discount'] = 0.1 instead.

Q275. [C] (Day 11: Pandas MultiIndex Cross-Section (.xs)) - .xs() (cross-section) extracts rows or columns at a particular level of a MultiIndex.

Q276. [A] (Day 11: Pandas Performance Hierarchy) - Vectorized operations (executed in compiled C/SIMD) are fastest (~1x). df.apply() incurs Python function call overhead (~10-50x slower). df.itertuples() yields lightweight namedtuples (~100x slower). df.iterrows() instantiates a heavy Series object per row (~1000x slower).

Q277. [A] (Day 11: Matplotlib Twin Axes (ax.twinx)) - ax.twinx() creates a new Axes object that shares the x-axis with ax, but displays its own independent scale on the right y-axis.

Q278. [D] (Day 11: Pandas Categorical Data Type) - Categoricals store repetitive strings as compact integer category codes and a small unique lookup table, dramatically saving RAM and accelerating groupbys.

Q279. [B] (Day 10: NumPy Axis Parameter in Aggregation) - Specifying axis=0 collapses the 0th dimension (rows) by summing down the columns, returning an array of shape (5,).

Q280. [B] (Day 11: Matplotlib Object-Oriented API vs pyplot) - The OO interface gives direct access to specific Figure and Axes instances, eliminating bugs caused by pyplot's implicit tracking of the 'current' active figure.

Q281. [A] (Day 01: Global Interpreter Lock (GIL) Concept) - CPython's GIL is a mutex designed to make thread-unsafe reference counting memory management safe, restricting pure Python CPU-bound execution to one thread at a time per process.

Q282. [C] (Day 02: String intern() Explicit Invocation) - sys.intern() registers the string in CPython's internal string table and returns the canonical interned object reference. Since both strings have identical content, both calls return the same object (s1 is s2 is True).

Q283. [C] (Day 03: Bisect Module for Sorted Sequences) - bisect_left uses binary search (O(log n)) to locate the insertion point that maintains sorted order. 25 belongs between 20 (index 1) and 30 (index 2), returning index 2.

Q284. [B] (Day 04: Dict Compact Layout in Python 3.6+) - The compact dict stores actual (hash, key, value) tuples consecutively in a dense array (which naturally preserves insertion order) and uses a small sparse hash index table containing integer offsets into the dense array.

Q285. [A] (Day 05: Function __closure__ Attribute) - Free variables captured by a closure are stored in cell objects inside the function's __closure__ attribute (a.__closure__[0].cell_contents is 10).

Q286. [D] (Day 06: Method Type Binding (Bound Method)) - When accessed on a class, it is a plain function object. When accessed on an instance, the descriptor protocol (__get__) binds the instance to produce a 'method' object that automatically passes p to self.

Q287. [B] (Day 06: Class Creation via type()) - The 3-argument signature of type is: type(name, bases_tuple, dict_of_attributes). Thus, type('User', (), {'role': 'admin'}) dynamically constructs the class.

Q288. [B] (Day 07: SQLite Foreign Key Constraints Enforcement) - In SQLite, foreign key enforcement is disabled by default for backward compatibility. Executing cursor.execute('PRAGMA foreign_keys = ON;') is required to enforce them.

Q289. [D] (Day 08: Coroutine Delegation Protocol) - Generators are lazy data producers evaluated with next(). 'async def' coroutines are concurrency constructs integrated with an event loop (asyncio) that pause execution at 'await' points until futures resolve.

Q290. [D] (Day 09: WSGI Specification Architecture) - WSGI (Web Server Gateway Interface) specifies that an application must be a callable accepting two parameters: environ (a dictionary of CGI environment variables) and start_response (a callback function taking status and headers).

Q291. [B] (Day 10: NumPy Vectorization vs CPython Loop) - NumPy arrays store raw primitive types in contiguous memory buffers. Operations are compiled C loops with SIMD vectorization. Standard Python lists store pointers to heap-allocated PyObject wrappers, causing heavy cache misses and dynamic type checking.

Q292. [B] (Day 11: Pandas merge() Types) - A left outer join retains all rows from the left DataFrame. If there is no matching key in the right DataFrame, right-table columns are populated with NaN (null).

Q293. [C] (Day 11: Pandas Indexing with .query()) - df.query() evaluates string expressions using NumExpr (improving speed on large datasets) and accepts standard boolean keywords 'and'/'or' without parentheses.

Q294. [D] (Day 10: NumPy newaxis Dimension Expansion) - np.newaxis (an alias for None) inserts a new axis of length 1 at the specified dimension, transforming a 1D array of shape (4,) into a 2D column vector of shape (4, 1).

Q295. [B] (Day 05: Argument Packing and Keyword Ordering) - Valid parameter order is: positional-only (optional) -> standard positional -> var-positional (*args) -> keyword-only (c=1) -> var-keyword (**kwargs). Option A matches this order.

Q296. [B] (Day 06: Method Overriding and super() Delegation) - Derived sets self.val = 2, and then calls super().__init__(), which overwrites self.val to 1. Output is 1.

Q297. [D] (Day 07: Text vs Binary Mode Cursor Offsets) - In text mode, character-to-byte mapping varies (e.g. variable-length UTF-8 encodings). Python forbids relative seeking with non-zero offsets in text mode to avoid landing in the middle of a multi-byte character.

Q298. [C] (Day 10: BeautifulSoup find vs select) - Both find(..., id='main') and select_one('#main') return the first matching Tag element, or None if no match is found.

Q299. [B] (Day 11: Pandas pivot_table() Aggregation) - df.pivot() reshapes data without aggregation and raises an error on duplicate index/columns. df.pivot_table() performs aggregation (default: np.mean) across duplicate entries.

Q300. [C] (Day 11: Matplotlib plt.subplots figsize Units) - Matplotlib figures are measured in inches: figsize=(width_in_inches, height_in_inches). Combined with dpi (dots per inch), it determines the final pixel resolution.